



جامعة محمد الأول بوجدة
UNIVERSITE MOHAMMED PREMIER OUJDA
ⵜⴰⵎⴰⵔⵜ ⵜⴰⵎⴰⵖⴻⵔⵜ ⵜⴰⵏⵓⵔⴼⴰⵏⵜ ⵜⴰⵖⵔⴼⴰⵏⵜ



المدرسة الوطنية للتكنولوجيا والرقمنة - بركان
ECOLE NATIONALE DE L'INTELLIGENCE ARTIFICIELLE ET DU DIGITAL - BERKANE
ⵎⴰⵎⴰⵔⴼⴰⵏⵜ ⵜⴰⵏⵓⵔⴼⴰⵏⵜ ⵜⴰⵖⵔⴼⴰⵏⵜ ⵜⴰⵎⴰⵖⴻⵔⵜ ⵜⴰⵏⵓⵔⴼⴰⵏⵜ - ⵎⴰⵎⴰⵔⴼⴰⵏⵜ

Université Mohamed Premier, Oujda, Maroc

École Nationale de l'Intelligence Artificielle et du Digital, Berkane

PROJET DE FIN D'ÉTUDES

Pour l'obtention du Diplôme d'Ingénieur d'État

Filière : Intelligence Artificielle et Digital

ADAS Test Case Generation using agentic AI

RÉALISÉ PAR

Ahmed Oukacha

Organisme d'accueil : Capgemini Engineering
Période de stage : Février 2026 – Aout 2026
Soutenu le : Juillet 2026

JURY DE SOUTENANCE

Nom du Professeur 1	Président du Jury	ENIAD
Nom du Professeur 2	Examineur	ENIAD
Nom de l'encadrant académique	Encadrant Académique	ENIAD
Nom de l'encadrant industriel	Relevant de l'organisme d'accueil	Capgemini Engineering

Remerciements

Au terme de ce projet de fin d'études, nous tenons à exprimer notre reconnaissance à celles et ceux qui ont jalonné notre parcours et contribué à la réussite de ce travail. Nos premiers remerciements s'adressent à Capgemini Engineering. Nous y avons trouvé bien plus qu'un lieu de stage : une véritable immersion dans un projet innovant et au cœur des technologies de pointe, alliant systèmes ADAS et intelligence artificielle.

Nous tenons à remercier chaleureusement nos encadrants pour leur accompagnement précieux. Leur disponibilité constante, leurs conseils avisés et la confiance qu'ils nous ont accordée ont été de véritables moteurs tout au long de cette expérience. Leur expertise technique et leur soutien ont été déterminants.

Travailler aux côtés des équipes de Capgemini Engineering a été un immense plaisir. Au-delà des tâches quotidiennes, ce sont les échanges partagés, les retours d'expérience et l'esprit de solidarité au sein de l'équipe qui ont enrichi notre vision professionnelle.

Nous tenons également à saluer l'École Nationale d'Intelligence Artificielle et du Digital de Berkane (ENIAD). Merci à l'ensemble du corps professoral pour la rigueur de la formation et la qualité des connaissances transmises, qui constituent aujourd'hui le socle de nos compétences.

Enfin, que toutes les personnes qui ont soutenu ce projet, de près ou de loin, trouvent ici l'expression de notre gratitude pour avoir rendu cette aventure humaine et professionnelle aussi mémorable.

Résumé

L'émergence des systèmes avancés d'aide à la conduite (ADAS) pose un défi majeur : la complexité croissante des processus de vérification et d'audit.

L'analyse des exigences, la conception des plans de test et la création des scénarios de test sont des tâches essentielles mais chronophages en ingénierie. C'est précisément là qu'intervient ce projet de fin d'études.

L'objectif est de concevoir une plateforme intelligente, basée sur une approche de agentic AI, pour assister les ingénieurs ADAS. Notre solution coordonne le travail de plusieurs agents spécialisés qui collaborent pour analyser les spécifications, structurer les données et produire des cas de test cohérents et traçables.

L'architecture exploite la puissance des grands modèles de langage LLM et utilise le framework LangGraph pour orchestrer les interactions entre agents. Une approche human-in-the-loop garantit que l'expert reste le décideur final.

Sur le plan technique, la solution repose sur FastAPI et Docker, avec un système de monitoring basé sur Grafana pour assurer la performance et la fiabilité.

Les résultats montrent que l'intégration de l'intelligence artificielle avec la supervision humaine permet d'améliorer significativement la productivité tout en garantissant des tests fiables et standardisés.

Mots-clés

[Agentic AI], [ADAS], [Large Language Models], [LangGraph], [Human-in-the-Loop], [Agent Memory], [Test Plan Generation], [Test Case Generation], [FastAPI], [Docker], [Grafana],

Abstract

The emergence of Advanced Driver Assistance Systems ADAS poses a major challenge: the increasing complexity of verification and audit processes.

Requirements analysis, test plan design, and test scenario creation are essential but time-consuming tasks in engineering. This is precisely where this final-year project comes in.

The goal is to design an intelligent platform, based on an agentic AI approach, to assist ADAS engineers. Our solution coordinates the work of several specialized agents that collaborate to analyze specifications, structure data, and produce consistent and traceable test cases.

The architecture leverages the power of large language models LLMs and uses the LangGraph framework to orchestrate interactions between agents. A human-in-the-loop approach ensures that the expert remains the final decision-maker.

From a technical standpoint, the solution is based on FastAPI and Docker, with a monitoring system based on Grafana to ensure performance and reliability.

The results show that integrating artificial intelligence with human supervision significantly improves productivity while ensuring reliable and standardized testing.

Keywords

[Agentic AI], [ADAS], [Large Language Models], [LangGraph], [Human-in-the-Loop], [Agent Memory], [Test Plan Generation], [Test Case Generation], [FastAPI], [Docker], [Grafana],

Abstract in Arabic

يُشكّل ظهور أنظمة مساعدة السائق المتقدمة (ADAS) تحديًا كبيرًا:

التعقيد المتزايد لعمليات التحقق والتدقيق.

يُعدّ تحليل المتطلبات، وتصميم خطة الاختبار، وإنشاء سيناريوهات الاختبار مهامًا أساسية ولكنها تستغرق وقتًا طويلاً في الهندسة. وهنا تحديدًا يأتي دور مشروع التخرج هذا.

الهدف هو تصميم منصة ذكية، تعتمد على نهج الذكاء الاصطناعي الوكيل، لمساعدة مهندسي أنظمة مساعدة السائق المتقدمة. يُستق حُلّا عمل العديد من الوكلاء المتخصصين الذين يتعاونون لتحليل المواصفات، وهيكلة البيانات، وإنتاج حالات اختبار متسقة وقابلة للتتبع.

تستفيد البنية من قوة نماذج اللغة الكبيرة (LLMs) وتستخدم إطار عمل LangGraph لتنظيم التفاعلات بين الوكلاء. ويضمن نهج التدخل البشري أن يظل الخبير هو صاحب القرار النهائي.

من الناحية التقنية، يعتمد الحلّ على FastAPI و Docker، مع نظام مراقبة قائم على Grafana لضمان الأداء والموثوقية. تُظهر النتائج أن دمج الذكاء الاصطناعي مع الإشراف البشري يُحسن الإنتاجية بشكل ملحوظ مع ضمان إجراء اختبارات موثوقة وموحدة.

الكلمات المفتاحية

[أدخل من 4 إلى 5 كلمات مفتاحية باللغة العربية مفصولة بفواصل]

Sommaire

Remerciements

.....	2
Résumé	3
Abstract	4
Abstract in Arabic	5
Introduction Générale	13
1. Chapitre 1 — Contexte général du projet	16
1.1 Présentation de Capgemini Engineering	16
1.1.1 Historique et positionnement	16
1.1.2 Offres des services du Capgemini Engineering	17
1.1.3 Domaine d'activité de Capgemini Engineering	17
1.1.4 Organisation de Capgemini Engineering au Maroc	18
1.1.5 Présentation de l'équipe SDA	19
1.2 Présentation du projet ADAS-R2T	20
1.2.1 Problématique	20
1.2.2 Objectifs du projet	21
1.2.3 Expression des besoins	22
1.3 Méthodologie de Travail	23
1.3.1 Management du projet	23
1.3.2 Approche de recherche	24
1.3.3 Planification du projet	25
2. Chapitre 2 — État de l'art	27
2.1 De l'IA Générative à l'IA Agentique	27
2.1.1 L'IA Générative	27
2.1.2 L'IA Agentique	29
2.1.3 Les composants d'un système d'IA Agentique	30
2.2 Patrons architecturaux des systèmes agentiques	31
2.2.1 Quand utiliser des agents AI et LLM Workflows	31
2.2.2 Modèles de conception d'agents d'IA et LLM Workflows	32
2.2.3 Agents autonomes	34
2.2.4 Choix architectural pour ADAS-R2T	34
2.3 LangGraph : framework d'orchestration	34
2.3.1 Pourquoi LangGraph	35
2.3.2 LangGraph vs LangChain ?	35
2.3.3 Complémentarité entre LangGraph et LangChain	36
2.3.4 Concepts fondamentaux	37
2.4 Ingénierie des prompts et du contexte	37
2.4.1 Du prompt engineering au context engineering	37
2.5 Travaux connexes	38
2.5.1 Validation ADAS/ADS et génération de scénarios conformes à SOTIF	38

2.5.2 Génération de scénarios à partir de descriptions textuelles	39
2.5.3 Apport des LLMs dans la génération de scénarios de test	40
2.5.4 Exploitation des sources ouvertes et OSINT pour les scénarios ADAS	41
2.5.5 Standards ASAM et interopérabilité des scénarios	41
2.5.6 Analyse comparative des travaux étudiés	42
2.5.7 Positionnement du projet ADAS-R2T	43
3. Chapitre 3 — Architecture Générale du Projet	45
3.1 Vue d'ensemble	45
3.1.1 Flux global	45
3.1.2 Les quatre étapes du "pipeline"	46
3.1.3 Les trois modes d'entrée	47
3.2 Architecture technique globale	47
3.2.1 Vue des composants	48
3.2.2 "Pipeline" d'orchestration "LangGraph"	48
3.2.3 Modèles de langage	48
3.2.4 Ingénierie des prompts	49
3.2.5 Mémoire et persistance ("PostgreSQL")	50
3.2.6 Observabilité ("Langfuse", "Prometheus", "Grafana")	50
3.2.7 Validation et évaluation	50
3.2.8 Choix techniques et justifications	51
3.3 Architecture du graphe agentique	51
3.3.1 Vue d'ensemble du graphe	51
3.3.2 Agent 1 : Extraction des entrées	52
3.3.3 Agent vidéo : analyse et mutations	52
3.3.4 Agent 2 : Analyse sémantique	54
3.3.5 Agent 3 : Génération des cas de test	55
3.3.6 Agent 4 : Évaluation et sortie	56
3.3.7 Parallélisme et contrôle de concurrence	57
3.3.8 Déroulement temporel d'une exécution	58
3.4 Architecture "HITL" et "Time Travel"	59
3.4.1 Le principe d'interruption	59
3.4.2 Les décisions de l'utilisateur	60
3.4.3 Régénération sélective par "Time Travel"	60
3.4.4 Régénération globale	61
3.4.5 Retour à HITL	61
3.5 Architecture mémoire	62
3.5.1 Les trois niveaux de mémoire	62
3.5.2 Flux d'écriture et de lecture	64
3.6 Architecture d'observabilité	65
3.6.1 Traçage LLM "Langfuse"	65
3.6.2 Métriques opérationnelles "Prometheus" et "Grafana"	65
3.6.3 Logging structuré "structlog"	66
3.7 Sécurité et résilience	66
3.7.1 Flux d'écriture et de lecture	67
3.8 Architecture d'observabilité	67
3.8.1 Traçage LLM "Langfuse"	67
3.8.2 Métriques opérationnelles "Prometheus" et "Grafana"	68
3.8.3 Logging structuré "structlog"	68
3.9 Sécurité et résilience	69

4. Chapitre 4 — Architecture Générale du Projet	71
4.1 Réalisation et implémentation	71
4.1.1 Organisation du code source	71
4.1.2 Principes d'organisation	72
4.1.3 Rôles des répertoires	73
4.1.4 Gestion des dépendances	73
4.2 Itérations de développement	74
4.2.1 MVP 1 : Requirements to Tests	74
4.2.2 MVP 2 : Video Input Layer	74
4.2.3 MVP 3 : Human in the Loop	75
4.2.4 MVP 4 : Chatbot	76
4.2.5 MVP 5 : Mémoire a long terme	76
4.2.6 Synthèse des itérations	77
4.3 Ingénierie des prompts	77
4.3.1 Pourquoi séparer les prompts du code	77
4.3.2 Structure d'un prompt	78
4.3.3 L'enrichissement du contexte	79
4.4 Déploiement	79
4.4.1 Le défi de la reproductibilité	80
4.4.2 Le Dockerfile du pipeline	80
4.4.3 L'orchestration des services	81
4.4.4 La gestion des variables d'environnement	81

Liste des figures

Figure 1	historique de Capgemini Engineering	16
Figure 2	Organisation de Capgemini Engineering	18
Figure 3	Organisation de la direction AIS	19
Figure 4	Signification de l'approche SDA	19
Figure 5	Limites du processus actuel de génération des tests ADAS	20
Figure 6	Diagramme de cas d'utilisation de la plateforme ADAS-R2T	22
Table 1	Liste des besoins fonctionnels	23
Table 2	Liste des besoins non fonctionnels	23
Figure 7	Diagramme de Gantt du projet ADAS-R2T	26
Figure 8	Comparaison des attributs structurels entre GenAI traditionnelle et IA agentique	27
Figure 9	Chaînage de prompts avec étape de validation	32
Figure 10	Workflow de routage vers des spécialistes LLM	33
Figure 11	Workflow de parallélisation avec fusion des résultats	33
Figure 12	Workflow Orchestrator-Workers	33
Figure 13	Workflow Evaluator-Optimizer	34
Table 3	Correspondance entre les patrons architecturaux et leur usage dans ADAS-R2T.	34
Table 4	Comparaison synthétique des travaux connexes liés à la génération de scénarios et de tests ADAS	43
Figure 14	Vue synthétique du pipeline ADAS-R2T	46
Figure 15	Les quatre étapes du pipeline de generation.	46
Table 5	Modes d'entrée et sorties générées par "ADAS-R2T"	47
Figure 16	Architecture globale des composants agentiques et techniques d'ADAS-R2T	48
Table 6	Choix technologiques retenus pour l'architecture ADAS -R2T	51
Figure 17	Vue globale du pipeline	51
Figure 18	Agent 1 Workflows	52
Figure 19	Agent video Workflows	53
Figure 20	Agent 3 Analyse sémantique	54
Figure 21	Agent 3 Workflows	56
Figure 22	Agent 4 Workfolows	57
Figure 23	Diagramme de séquence du flux de génération Excel dans ADAS-R2T	58
Figure 24	Revue HITL — Round 1 et régénération sélective	59
Figure 25	Revue HITL — approbation finale et génération du fichier Excel	60
Figure 26	Revue HITL — réentrée optionnelle dans le cycle de revue	62
Figure 27	Architecture memoire	63
Table 7	Limites de requêtes appliquées aux points d'accès de l'"API"	69
Figure 28	Diagramme de séquence du flux de génération Excel dans ADAS-R2T	70
Listing 1	Arborescence du projet ADAS-R2T	72
Table 8	Organisation logique des principaux répertoires du projet ADAS-R2T	73
Table 9	Comparaison synthétique des versions MVP du projet ADAS-R2T	77
Table 10	Couches d'information exploitées par le pipeline ADAS-R2T	79

Table 11	Services Docker utilisés pour le déploiement et la supervision d'ADAS-R2T	81
Table 12	Familles de variables d'environnement utilisées par ADAS-R2T	82

Liste des tableaux

Table 1	Liste des besoins fonctionnels	23
Table 2	Liste des besoins non fonctionnels	23
Table 3	Correspondance entre les patrons architecturaux et leur usage dans ADAS-R2T.	34
Table 4	Comparaison synthétique des travaux connexes liés à la génération de scénarios et de tests ADAS	43
Table 5	Modes d'entrée et sorties générées par "ADAS-R2T"	47
Table 6	Choix technologiques retenus pour l'architecture ADAS-R2T	51
Table 7	Limites de requêtes appliquées aux points d'accès de l'"API"	69
Table 8	Organisation logique des principaux répertoires du projet ADAS-R2T	73
Table 9	Comparaison synthétique des versions MVP du projet ADAS-R2T	77
Table 10	Couches d'information exploitées par le pipeline ADAS-R2T	79
Table 11	Services Docker utilisés pour le déploiement et la supervision d'ADAS-R2T	81
Table 12	Familles de variables d'environnement utilisées par ADAS-R2T	82

Liste d'abréviations

ACC	Adaptive Cruise Control — Régulateur de vitesse adaptatif
ADAS	Advanced Driver Assistance Systems — Systèmes avancés d'aide à la conduite
AEB	Automatic Emergency Braking — Freinage d'urgence automatique
API	Application Programming Interface — Interface de programmation applicative
BSW	Blind Spot Warning — Avertissement d'angle mort
CDC	Cahier des charges — Document de spécification du projet
CI/CD	Continuous Integration / Continuous Deployment — Intégration et déploiement continus
ESC	Electronic Stability Control — Contrôle électronique de stabilité
FCW	Forward Collision Warning — Avertissement de collision frontale
GenAI	Generative Artificial Intelligence — Intelligence artificielle générative
HMI	Human-Machine Interface — Interface homme-machine
JWT	JSON Web Token — Jeton d'authentification web
LKA	Lane Keeping Assist — Aide au maintien de voie
LLM	Large Language Model — Grand modèle de langage

Introduction Générale

L'industrie automobile traverse actuellement une période charnière, marquée par la transformation des véhicules en plateformes mobiles intelligentes. Au cœur de cette révolution se trouvent les systèmes avancés d'aide à la conduite (ADAS), piliers indispensables pour redéfinir la sécurité routière et offrir une expérience de conduite plus confortable et plus sûre.

Ces systèmes englobent un large éventail de technologies essentielles, du régulateur de vitesse adaptatif (ACC) et du freinage d'urgence automatique (AEB) à l'assistance au maintien de voie (LKA), à l'alerte de collision frontale (FCW) et à la reconnaissance des panneaux de signalisation (TSR). Malgré leurs fonctions variées, leur objectif est unique : analyser l'environnement du véhicule, interpréter les situations de conduite et intervenir en temps opportun pour protéger le conducteur.

Cependant, cette intelligence représente un défi d'ingénierie : la complexité fonctionnelle de ces systèmes croît rapidement. Chaque fonction ADAS est régie par un cahier des charges technique rigoureux qui définit précisément les conditions d'activation, les seuils de réponse, les changements d'état, les messages d'avertissement et les contraintes temporelles. Ceci souligne l'importance cruciale de la phase de validation dans le cycle de développement du véhicule. Sans elle, la sécurité et la fiabilité de ces fonctions critiques ne peuvent être garanties.

En réalité, l'élaboration de plans et de scénarios de test est la pierre angulaire des équipes de vérification. C'est une tâche exigeante qui requiert une analyse minutieuse de chaque exigence fonctionnelle afin de couvrir les scénarios nominaux, les situations critiques et les conditions rares. Cependant, ce processus reste largement manuel dans de nombreux environnements industriels, ce qui le rend chronophage, dépendant de l'expertise individuelle des ingénieurs et sujet aux erreurs d'interprétation ou à une couverture incomplète des exigences.

C'est pourquoi les technologies d'intelligence artificielle, et plus particulièrement les grands modèles de langage (LLM), représentent une option prometteuse pour le développement et l'automatisation des processus de test, grâce à leur capacité d'analyse du langage naturel et d'extraction de données structurées. Mais dans un secteur aussi sensible que l'automobile, la

simple génération de texte ne suffit pas ; il est nécessaire de disposer d'un environnement de travail garantissant un suivi, une correction et une traçabilité rigoureux des résultats.

Dans cette optique, notre projet de fin d'études, conçu au sein de Capgemini Engineering, vise à construire et développer une plateforme intelligente que nous avons nommée ADAS-R2T (Requirements to Tests). La plateforme vise à assister les ingénieurs ADAS en convertissant automatiquement les exigences fonctionnelles en plans de test et cas de test structurés, cohérents et traçables.

Notre solution repose sur une approche d'IA multi-agents, où le travail est réparti entre un réseau d'agents numériques spécialisés. Chaque agent logiciel prend en charge une tâche spécifique, depuis l'analyse des exigences et l'extraction d'indicateurs jusqu'à la formulation des plans de test, l'évaluation de la couverture et l'amélioration des résultats. Cette approche dépasse la génération traditionnelle et s'inscrit dans une logique de planification, d'évaluation et d'auto-correction.

Sur le plan structurel, ces modèles de langage sont intégrés au framework LangGraph pour gérer le dialogue et la coordination entre les agents. Nous avons également intégré le principe de Human-in-the-Loop afin que la décision finale revienne à l'expert humain. L'objectif n'est pas de remplacer l'ingénieur, mais de lui fournir un outil intelligent qui lui permette de gagner du temps tout en assurant un suivi précis entre l'exigence initiale et le test final généré.

Techniquement, la plateforme repose sur une architecture logicielle flexible et évolutive, s'appuyant sur les services backend FastAPI et fonctionnant dans des conteneurs Docker pour faciliter son déploiement en milieu industriel. Elle est également optimisée par un système de surveillance et d'analyse continue des performances. La valeur ajoutée de ce travail réside dans la création d'un parcours intelligent intégré pour l'automatisation des tests ADAS, l'augmentation de la couverture fonctionnelle et la garantie d'un suivi complet des exigences, tout en préservant le rôle de la supervision humaine à l'ère de l'intelligence artificielle.

Le présent rapport décrit le travail réalisé au sein de **Capgemini Engineering**.

Le présent mémoire est structuré comme suit :

- **Chapitre 1** : Contexte général du projet, présentation de l'organisme d'accueil, problématique, cahier des charges et méthodologie de travail.
- **Chapitre 2** : État de l'art relatif aux systèmes ADAS, à l'intelligence artificielle générative, aux grands modèles de langage et aux architectures agentiques.

- **Chapitre 3** : Présentation de l'architecture globale de la plateforme ADAS-R2T.
- **Chapitre 4** : Implémentation technique de la solution.
- **Chapitre 5** : Évaluation des résultats obtenus, limites et perspectives.

Contexte général du projet

1.1 Présentation de Capgemini Engineering

1.1.1 Historique et positionnement

Capgemini et Altran ont annoncé, le 24 juin 2019, un accord portant sur l'acquisition de la société Altran par Capgemini. Le 1er avril 2020, l'OPA amicale de Capgemini sur Altran a été finalisée. Dominique Cerutti, directeur général d'Altran, a confirmé que cette acquisition allait créer un leader mondial de l'industrie intelligente au service de la transformation numérique des entreprises. En avril 2021, Altran est devenue **Capgemini Engineering**.

Figure 1 présente l'évolution historique de Capgemini Engineering au Maroc:

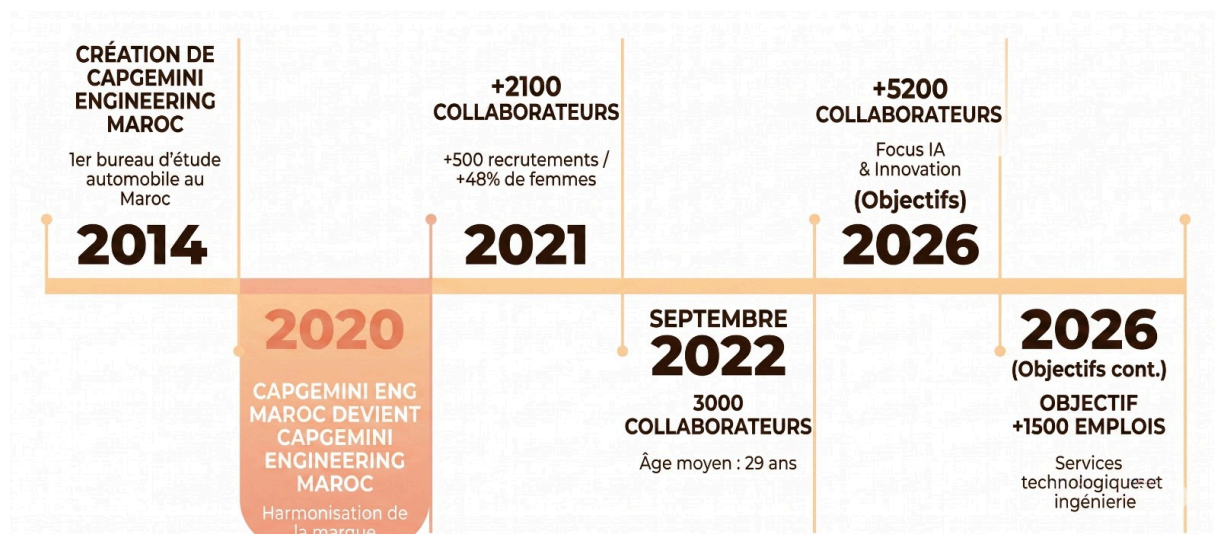


Figure 1: historique de Capgemini Engineering

1.1.2 Offres des services du Capgemini Engineering

Les offres du groupe suivent l'ensemble du cycle de RD : conception, développement, test, innovation et prototypage, et accompagne également l'industrialisation, le service après-vente et la production. Elle est caractérisée avec son fort et unique savoir-faire en matière d'innovation, Capgemini Engineering répond aux besoins de ses clients dans 6 catégories d'activités : - **Consulting** : Accompagne les clients du Groupe dans la transformation de leurs opérations, les conseille dans la définition de leurs stratégies en matière d'innovation et de leurs services et produits futurs.

- **Digital** : Accompagne les clients dans leur transformation digitale par le biais de la capitalisation sur sa connaissance de leurs produits et processus industriels, et sur l'expertise de ses ingénieurs spécialisés dans les métiers du numérique.

- **Engineering** : aide les clients du Groupe dans le développement de nouveaux produits et système tout en réduisant leurs délais de mise sur le marché et leurs coûts, et les accompagne dans l'amélioration de leurs processus industriels et leurs systèmes de production.

- **World Class Centers** : propose les solutions et les services dans des domaines de pointe en s'appuyant sur sept centres d'expertise mondiaux regroupant les investissements et actifs du Groupe correspondant.

- **Industrialized Global Shore** : permet aux clients de bénéficier d'une expertise globale et de réunir la compétitivité et les normes de qualité les plus élevés. Cette solution industrielle de prestations de services d'ingénierie et de RD du Groupe repose sur 5 centres d'ingénierie mondiaux, situés Near- et offshore.

- **Cambridge Consultants** : spécialisé dans le développement de produits innovants, accompagné par des équipes scientifiques de haut niveau, et s'appuyant sur des laboratoires dédiés aux États-Unis et Royaume-Uni.

1.1.3 Domaine d'activité de Capgemini Engineering

Grâce à une maîtrise avancée des technologies digitales et logicielles, l'entreprise joue un rôle clé dans la transformation des industries vers l'Intelligent Industry. Avec plus de 55 000

ingénieurs et scientifiques répartis dans plus de 50 pays, Capgemini Engineering intervient dans des secteurs variés tels que :

- **Secteur automobile** : elle travaille principalement STELLANTIS (Ex. PSA), Le constructeur automobile VW et BOSCH.
- **Secteur aéronautique** : SAFRAN, AIRBUS, DASSAULT AVIATION.
- **Secteur ferroviaire** : ALSTOM, SNCF, BOMBARDIER.
- **Secteur Sciences de la vie** : SANOFI, GSK.

1.1.4 Organisation de Capgemini Engineering au Maroc

Capgemini Engineering Maroc adopte une structure organisationnelle articulée autour de quatre grands pôles. Cette structuration vise à assurer une gestion optimale des projets, tout en favorisant l'expertise et la spécialisation technique [Figure 2].

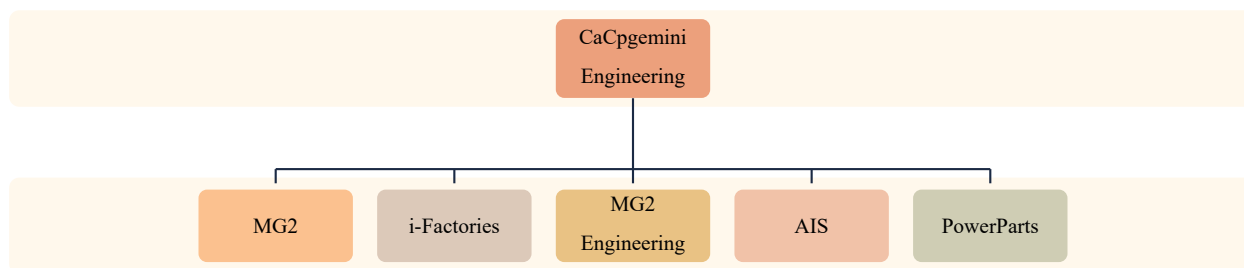


Figure 2: Organisation de Capgemini Engineering

Notre stage a été effectué au sein de la division Global Engineering Unit Morocco, un pôle stratégique regroupant plusieurs unités d'ingénierie spécialisées. Nous avons été intégrés à l'Advanced Intelligent Systems Engineering Unit (**AIS**), dirigée par M. Moulay El Ghil Hamdouchi. Cette unité est en charge du développement de solutions intelligentes pour divers secteurs, notamment l'automobile, en s'appuyant sur des approches innovantes telles que les systèmes embarqués, l'ingénierie des systèmes et les technologies avancées. Plus précisément, nous avons rejoint le département **EE ARCHITECTURE & SAFETY** au sein de l'équipe **SDA** [Figure 3].

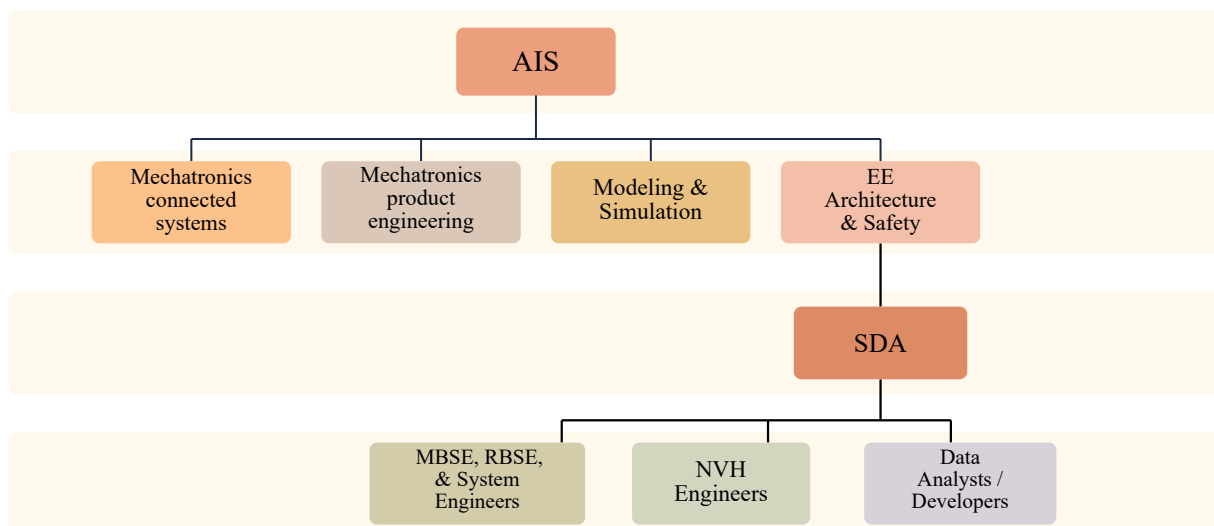


Figure 3: Organisation de la direction AIS

Dans cette organisation, notre travail s'inscrit plus particulièrement dans le sous-groupe **MBSE, RBSE, & System Engineers**, dont les activités sont liées à l'ingénierie des systèmes, à la modélisation des exigences, à la structuration des données techniques et à l'amélioration des processus de validation. Ce positionnement nous a permis de travailler dans un environnement fortement orienté vers les systèmes automobiles intelligents et les méthodologies d'ingénierie avancées. Il constitue ainsi un cadre adapté pour le développement de notre projet, qui vise à assister les ingénieurs dans la transformation des exigences fonctionnelles **ADAS** en plans et cas de test structurés, traçables et exploitables.

1.1.5 Présentation de l'équipe SDA

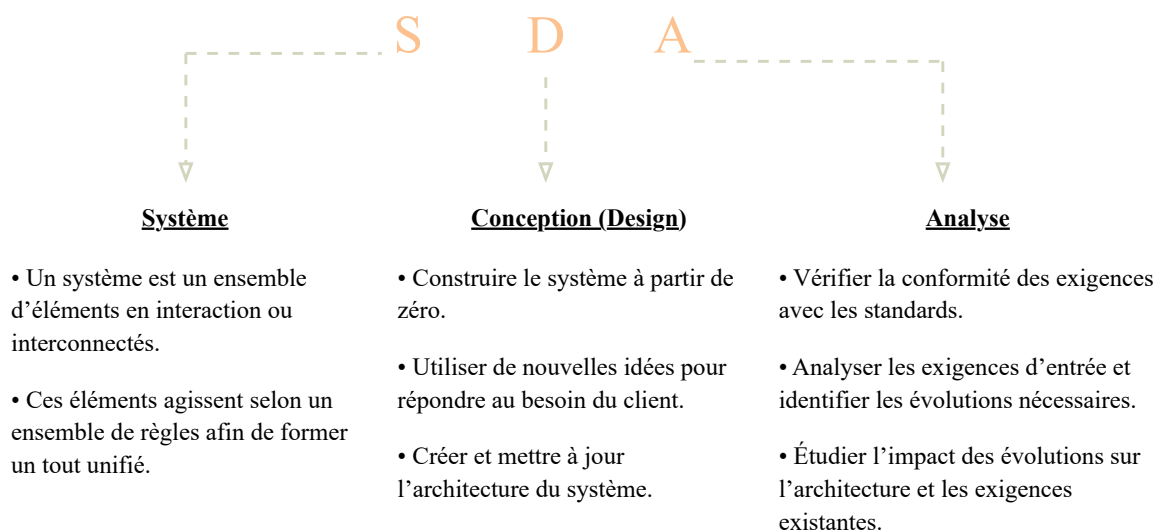


Figure 4: Signification de l'approche SDA

1.2 Présentation du projet ADAS-R2T

1.2.1 Problématique

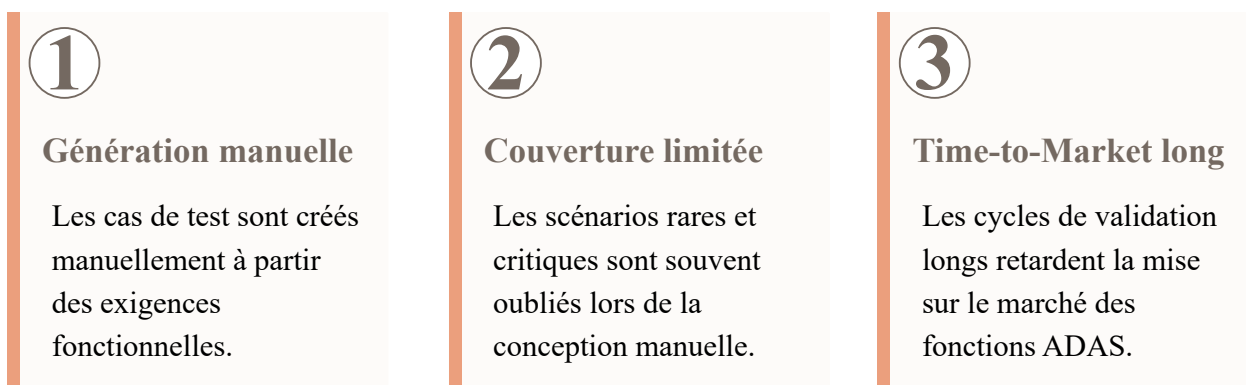


Figure 5: Limites du processus actuel de génération des tests ADAS

L'approche actuelle de conception et de développement des tests **ADAS** présente plusieurs obstacles structurels qui ont rendu cette recherche indispensable. Les principaux défauts peuvent être résumés comme suit :

- **Charge de traitement manuel** : La formulation des cas de test repose entièrement sur le travail manuel des ingénieurs de vérification, basé sur leur interprétation subjective des exigences fonctionnelles. Cette approche est non seulement chronophage, mais elle est aussi source d'erreurs d'interprétation et rend difficile la réplication des tests avec la même précision.
- **Lacunes de couverture fonctionnelle** : Tant que la conception est gérée manuellement, l'accent est naturellement mis sur les scénarios nominaux et idéaux, au détriment des cas limites, des scénarios défavorables et des conditions rares et complexes. Paradoxalement, c'est précisément dans ces environnements négligés que se cachent les erreurs les plus graves, celles qui ont le plus grand impact sur la sécurité des passagers.
- **Absence de traçabilité systématique** : Le système ne dispose pas d'un lien structurel clair et documenté entre l'exigence fonctionnelle initiale et les cas de test qui en découlent. Cette fragmentation rend l'évaluation de la couverture ou la mise à jour des ensembles de tests suite à une modification technique complexe et aléatoire.

- **Lenteur du rythme de production et des flux de travail** : L'ensemble du processus, de la réception des exigences à l'approbation de la version finale de test, prend plusieurs semaines, engendrant des goulots d'étranglement opérationnels qui retardent le déploiement sur le marché des fonctionnalités ADAS.
- **Données de conduite réelles inutilisées** : Bien que les entreprises possèdent des milliers d'heures d'enregistrements vidéo issus de véritables expériences de conduite sur route, cette mine de données en situation réelle reste inexploitée, ne permettant pas d'améliorer la qualité et la portée des tests. Compte tenu de ces facteurs, la problématique centrale de ce travail peut se résumer à la question suivante :

✓ NOTE

Comment concevoir un processus automatisé capable de traduire les exigences fonctionnelles des systèmes ADAS, rédigées en langage naturel, en cas de test précis, sans compromettre l'exhaustivité, tout en garantissant un suivi rigoureux et des normes de qualité conformes aux exigences de l'industrie automobile ? Autrement dit, et plus fondamentalement : comment faire confiance à l'intelligence artificielle générative pour automatiser l'inspection des systèmes critiques pour la sécurité et assurer la surveillance des cas rares et limites, sans que ce système ne devienne une « boîte noire » opaque, dépourvue de tout contrôle humain ?

1.2.2 Objectifs du projet

À cette fin, la feuille de route du projet “ *Test Cases Generator* ” a été conçue pour se concentrer sur la réalisation des objectifs stratégiques suivants :

- **Flux de travail entièrement automatisé(End To End)** : Nous visons à construire un cycle de traitement intégré qui démarre automatiquement dès l'importation du fichier d'exigences source (Excel) et se poursuit sans interruption jusqu'à la génération et l'extraction, dans le même format, du fichier de résultats structurés contenant les cas de test finaux.
- **Couverture complète et multidimensionnelle** : La plateforme ne se limitera pas à une analyse superficielle, mais est conçue pour décomposer chaque condition fonctionnelle selon cinq dimensions techniques (telles que les transitions d'état, les contraintes temporelles et les messages IHM). Cette approche garantit une génération structurée couvrant quatre catégories de tests : de routine, de pointe, inversés et rares.

- **Traçabilité rigoureuse** : Nous visons à établir un lien structurel indissociable entre chaque cas de test et sa source originale dans le cahier des charges en attribuant des identifiants uniques (identifiants uniques) permettant un audit inversé à tout moment.
- **Inspection intelligente par analyse vidéo** : L'un de nos principaux objectifs est de s'affranchir de la rigidité du texte en intégrant des scénarios extraits d'enregistrements de conduite réels. Cela confère aux tests un réalisme que les exigences théoriques seules ne peuvent atteindre.
- **Conception d'une architecture flexible et indépendante des fournisseurs** : nous avons conçu le système avec une architecture logicielle flexible qui lui permet de s'intégrer et de fonctionner de manière transparente avec divers fournisseurs de modèles de langage (tels que OpenAI, Gemini, ou même des modèles locaux via Ollama) sans qu'il soit nécessaire de réécrire ou de modifier le code source.

1.2.3 Expression des besoins

Dans le cadre de ce stage, les besoins suivants ont été identifiés en collaboration avec l'équipe encadrante. Pour mieux représenter les interactions entre les utilisateurs et la plateforme ADAS-R2T, un diagramme de cas d'utilisation a été réalisé. Il met en évidence les principales fonctionnalités du système ainsi que les acteurs concernés.

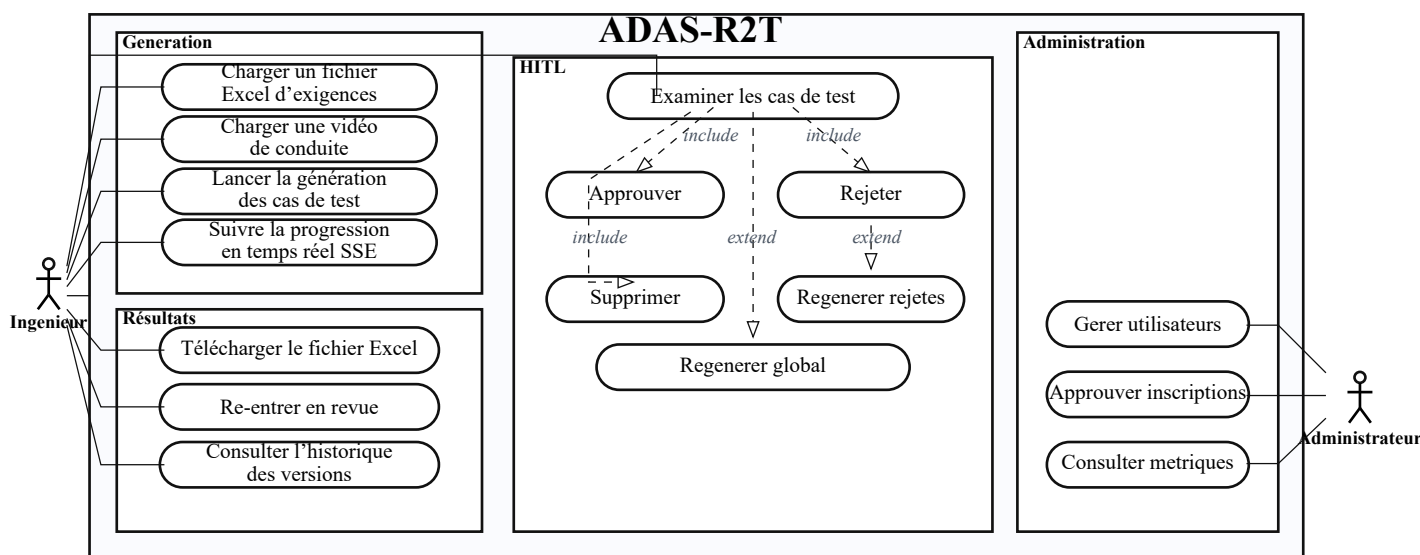


Figure 6: Diagramme de cas d'utilisation de la plateforme ADAS-R2T

• Besoins fonctionnels

ID	Description du besoin fonctionnel
BF01	Le système doit accepter un fichier Excel contenant des exigences fonctionnelles ADAS et générer un fichier Excel des cas de test.
BF02	Le système doit accepter une vidéo de conduite et extraire des scénarios de test avec raisonnement causal : cause, effet et conséquence.
BF03	Le système doit supporter trois modes d'entrée : Excel seul, vidéo seule, et Excel + vidéo.
BF04	Le système doit permettre à l'utilisateur de revoir les résultats avant le téléchargement : approbation, rejet avec feedback ou suppression.
BF05	Le système doit régénérer uniquement les cas de test rejetés sans relancer tout le pipeline.
BF06	Le système doit afficher la progression en temps réel pendant la génération à travers un mécanisme de streaming SSE.
BF07	Le système doit maintenir un historique des versions et des révisions : v1, v2, v3, etc.
BF08	Le système doit évaluer automatiquement 100 % des cas de test générés afin de détecter les contradictions, les éléments hors périmètre et les doublons.
BF09	Le système doit apprendre des feedbacks utilisateurs à travers une mémoire à long terme, incluant des règles partagées et des préférences personnelles.
BF10	Le système doit supporter plusieurs utilisateurs simultanément avec isolation des données.

Table 1: Liste des besoins fonctionnels

• Besoins non fonctionnels

ID	Catégorie	Description du besoin non fonctionnel
BNF01	Performance	La génération doit s'effectuer en moins de 120 secondes pour 1 exigence.
BNF02	Scalabilité	L'architecture doit supporter l'ajout des nouvelles fonctions ADAS sans modification majeure.
BNF03	Disponibilité	Le système doit reprendre après un crash sans perte des données.
BNF04	Sécurité	Les checkpoints doivent être chiffrés. L'authentification par clé API est obligatoire.
BNF05	Maintenabilité	Le code doit être modulaire, documenté et accompagné d'un logging structuré.
BNF06	Portabilité	Le système doit être déployable sur tout environnement.
BNF07	Interopérabilité	Le système doit communiquer via une API REST.

Table 2: Liste des besoins non fonctionnels

1.3 Méthodologie de Travail

1.3.1 Management du projet

Concernant la gestion de projet, nous avons opté pour la méthodologie Agile Scrum en raison de sa grande flexibilité, qui facilite la communication et simplifie la coordination quotidienne. Bien que cette méthodologie ait été initialement conçue pour des équipes plus importantes, nous l'avons adaptée avec succès à notre fonctionnement en duo (Binôme). Cette approche nous a permis de nous adapter rapidement aux évolutions techniques et de mener à bien les tâches sans tomber dans l'imprévisibilité, garantissant ainsi la livraison d'un produit de qualité conforme aux attentes. Pour concrétiser cette vision, nous avons adopté un ensemble de bonnes pratiques :

- **Définition des fonctionnalités et structuration du projet** : Nous avons commencé par définir les fonctionnalités requises à partir d'une analyse des besoins techniques, puis nous avons divisé le projet en périodes spécifiques **sprints** et en livrables minimums **MVP**.
- **Décomposition des tâches** : Chaque sprint a été décomposé en tâches plus petites et détaillées, précisément réparties pour faciliter le suivi quotidien.
- **Communication quotidienne avec le superviseur** : Nous avons veillé à avoir une brève discussion quotidienne avec le superviseur pour le tenir informé de l'avancement du travail et nous assurer que nous étions sur la bonne voie.
- **Réunion hebdomadaire prolongée (tous les mardis)** : Nous organisons une réunion régulière avec tous les stagiaires de l'équipe. Cette réunion permet d'échanger des points de vue, de faire le point sur l'avancement de chaque projet, d'aborder les difficultés techniques courantes et de définir les prochaines étapes.
- **Séance d'auto-évaluation (tous les vendredis)** : Nous tenons une réunion de clôture pour la semaine, consacrée aux trois questions essentielles de la méthodologie Scrum :
 - Quelles sont les tâches effectuées ?
 - Quelles sont les difficultés rencontrées ?
 - Quelles sont les tâches futures à réaliser ?
- **Une réunion mensuelle** où nous essayons de montrer l'importance de notre projet et l'efficacité de nos solutions.

1.3.2 Approche de recherche

Pour inscrire ce travail dans une perspective scientifique, nous avons identifié la méthodologie de la Recherche en Sciences de la Conception **DSR** comme le cadre et le guide idéal pour notre projet. Ce choix découle de la nature même de cette méthodologie : elle constitue l'option la plus

appropriée lorsque l'objectif est de créer des solutions pratiques à des problèmes réels et complexes, grâce à un processus itératif alternant conception, construction et évaluation continue de l'impact technique ou logiciel **artefact**. Contrairement aux approches traditionnelles qui se contentent d'observer ou d'interpréter des phénomènes, la **DSR** se concentre sur la conception d'outils fonctionnels utilisables et mesurables sur le terrain. C'est précisément ce qui s'applique à notre projet, où l'impact logiciel représente ici un flux de travail dynamique **Workflow** que nous avons construit à partir de LangGraph afin de générer des tests ADAS de manière structurée et évolutive. La méthodologie **DSR** a été mise en œuvre dans notre projet à travers deux cycles successifs de développement et d'évaluation :

- le premier cycle, axé sur l'établissement du noyau de base du flux de travail, a consisté à définir les composants essentiels du système, à ajuster les mécanismes de réception des données d'entrée, à concevoir la logique de traitement et à définir la structure d'état qui régit le graphe.
- Le second cycle, consacré au raffinement et à l'amélioration, a porté sur le renforcement des mécanismes d'auto-vérification et d'autocontrôle au sein du graphe, ainsi que sur le renforcement de la logique de génération afin d'améliorer la qualité et la fiabilité des résultats finaux. Cette progression itérative illustre parfaitement la philosophie **DSR**, fondée sur la triade « Construire, Évaluer, Améliorer en continu ».

Enfin, la conception de cette solution n'était pas accidentelle, mais plutôt basée sur une double base de connaissances : un aspect technique lié à la physique de la construction de systèmes basés sur des graphes et des sorties régies par des états (Stateful Systems) dans l'environnement LangGraph, et un aspect industriel conforme aux exigences strictes et précises des tests de systèmes ADAS dans le secteur automobile.

1.3.3 Planification du projet

Afin de garantir le respect du calendrier de formation et une gestion efficace du temps, le projet a fait l'objet d'une planification par phases rigoureuse. Nous avons décomposé la feuille de route en tâches et sous-tâches plus petits planifiées à l'aide de **Notion app**, directement liés à chaque version des livrables initiaux **MVP**. Le diagramme suivant résume la séquence chronologique et les interrelations structurelles de ces tâches tout au long du projet :

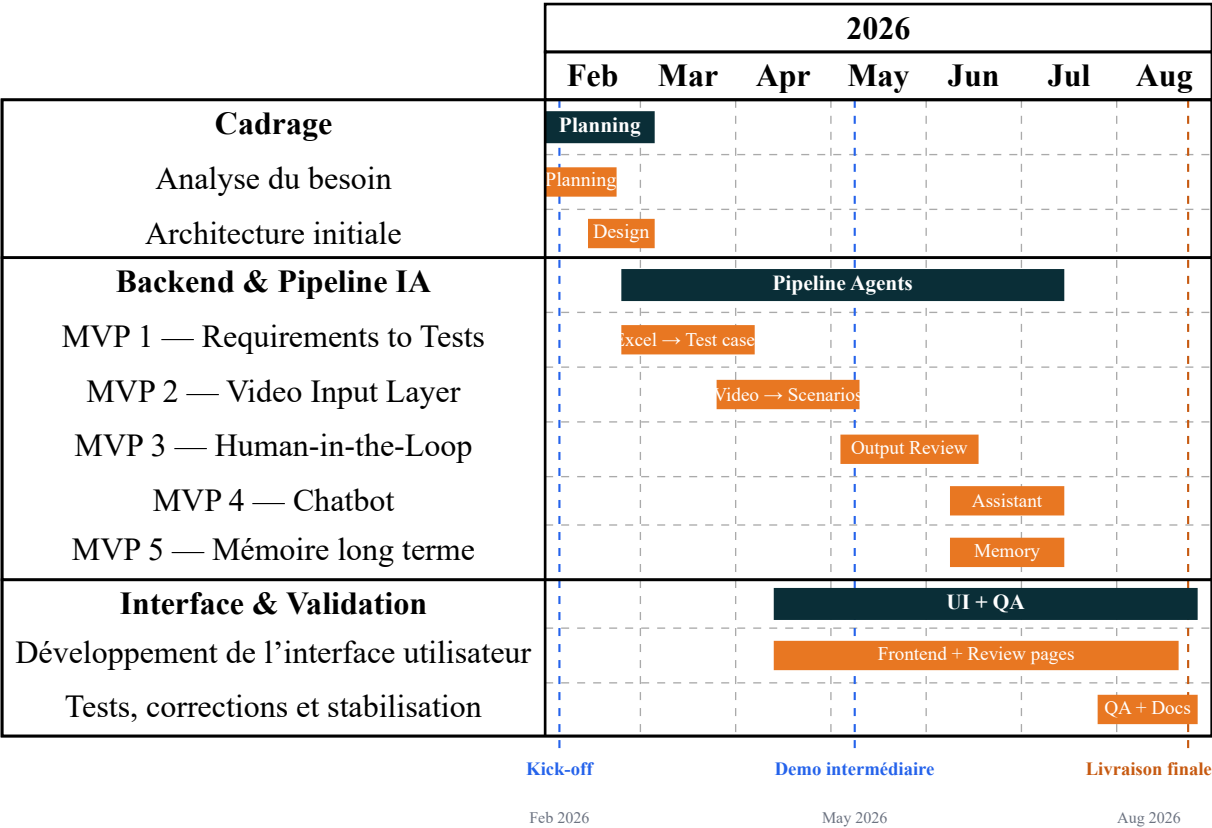


Figure 7: Diagramme de Gantt du projet ADAS-R2T

État de l'art

2.1 De l'IA Générative à l'IA Agentique

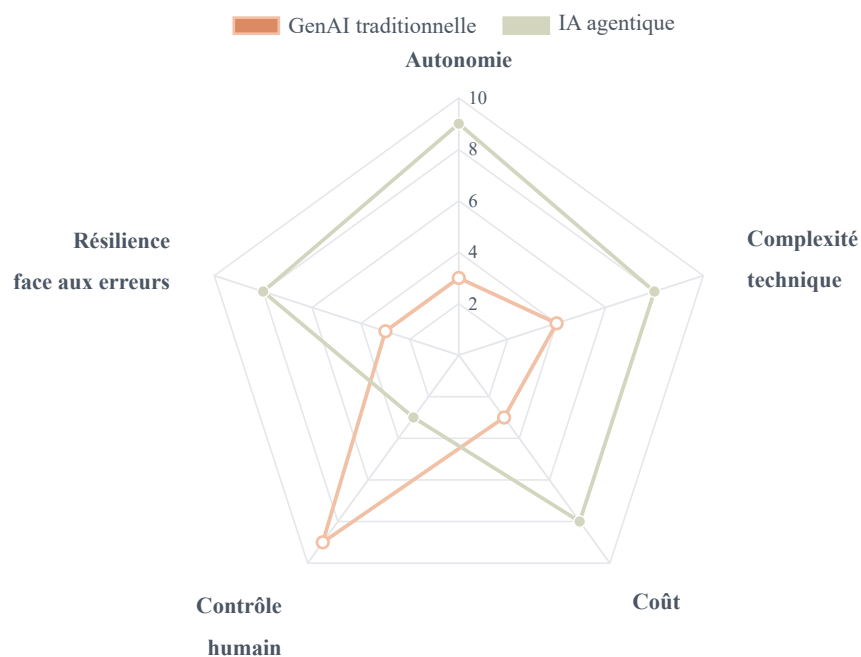


Figure 8: Comparaison des attributs structurels entre GenAI traditionnelle et IA agentique

2.1.1 L'IA Générative

L'intelligence artificielle générative constitue une rupture importante par rapport aux approches classiques de l'IA. Les modèles traditionnels étaient principalement conçus pour accomplir des tâches d'analyse, de classification ou de prédiction, comme reconnaître un objet dans une image ou estimer une valeur à partir de données existantes. Les modèles génératifs intro-

duisent une logique différente. Au lieu de se limiter à l'identification de catégories ou à la production de résultats prédictifs, ils apprennent les régularités profondes présentes dans les données d'entraînement. Cette capacité leur permet ensuite de générer de nouveaux contenus, qu'il s'agisse de texte, d'images, de code, d'audio ou d'autres formats numériques. Les grands modèles de langage, tels que **GPT-4, Claude ou Gemini**, représentent l'une des applications les plus visibles de cette évolution. Ils peuvent produire des documents structurés, résumer des textes volumineux, traduire entre plusieurs langues ou encore générer du code informatique.

Toutefois, malgré leurs performances, ces modèles restent limités par une logique fondamentalement réactive : ils répondent à une consigne, produisent une sortie, puis interrompent leur action. Ils ne disposent donc pas, par défaut, d'une capacité d'initiative autonome. Ils ne surveillent pas continuellement les résultats obtenus, ne planifient pas spontanément une suite

d'actions et ne corrigent pas leurs erreurs sans nouvelle intervention humaine. Cette limite explique l'émergence progressive des agents intelligents, qui cherchent à dépasser le simple modèle conversationnel pour aller vers des systèmes capables d'observer, décider et agir de manière plus autonome.

2.1.2 L'IA Agentique

L'IA Agentique représente l'étape suivante : le passage de la *création de contenu* à l'*exécution d'objectifs*. Un agent IA ne reçoit pas une liste de tâches séquentielles — il reçoit un objectif de haut niveau et conduit autonomement le processus pour l'atteindre.

Cette transition repose sur quatre piliers fondamentaux :

- **L'autonomie** : L'autonomie : l'agent est capable de prendre certaines décisions et d'exécuter des actions sans attendre une instruction humaine à chaque étape. Cette autonomie peut concerner l'exécution des tâches, le choix des actions ou encore l'utilisation d'outils externes. Elle doit cependant rester encadrée par des règles, des autorisations et, lorsque c'est nécessaire, une validation humaine.
- **Goal Oriented** : l'agent travaille à partir d'un objectif persistant. Cet objectif sert de direction principale à toutes ses actions. Par exemple, si l'objectif est de recruter un profil technique, l'agent garde ce but en mémoire et organise ses décisions autour de celui-ci, tout en respectant les contraintes définies comme le budget, les compétences demandées ou les délais.
- **La planification** : l'agent décompose un objectif complexe en étapes plus simples et plus concrètes. Il peut proposer plusieurs plans possibles, comparer leurs avantages et leurs limites, puis sélectionner la stratégie la plus adaptée selon les ressources disponibles, les risques, les coûts et les contraintes de départ.
- **Le raisonnement** : l'agent ne se limite pas à exécuter des actions. Il analyse les informations, compare les options, interprète les résultats intermédiaires et choisit les décisions les plus pertinentes.
- **L'adaptabilité** : Lorsqu'un événement imprévu survient ou qu'une stratégie ne donne pas les résultats escomptés, l'agent peut adapter son plan. Il peut modifier sa tactique, proposer une alternative ou solliciter une intervention humaine. L'important est de rester fidèle à l'objectif principal, même face à l'évolution de la situation.

- **Context Awareness** : Le système conserve et exploite les informations importantes tout au long du processus. Il prend en compte l'objectif initial, les actions réalisées, les préférences de l'utilisateur, les conditions environnementales, les réactions des outils et les règles à suivre. Cette mémoire contextuelle permet d'éviter les répétitions et de maintenir la cohérence logique entre les différentes étapes.

✓ NOTE

L'IA Générative est une **capacité** ; la faculté de raisonner et créer. L'IA Agentique est un **comportement** ; la capacité d'utiliser cette faculté comme moteur d'exécution. Le LLM n'est pas remplacé : il est *enveloppé* de mémoire, d'outils et d'un planificateur.

2.1.3 Les composants d'un système d'IA Agentique

Les composants d'un système d'IA agentique Pour comprendre le fonctionnement d'un agent IA, il faut dépasser l'idée d'un simple modèle de langage qui génère des réponses. Un système agentique repose sur une architecture plus riche, composée de plusieurs éléments qui travaillent ensemble. Chacun joue un rôle précis : comprendre l'objectif, organiser les actions, utiliser les bons outils, conserver le contexte et maintenir le système sous contrôle.

On peut généralement distinguer cinq composants principaux : **le Brain, l'Orchestrator, les Tools, la Memory et le Supervisor.** [1]

- **Le Brain** : ou cerveau de l'agent, est la partie qui interprète la demande de l'utilisateur. Il transforme une instruction parfois générale ou ambiguë en objectif clair et exploitable. C'est à ce niveau que l'agent commence à comprendre la situation, à analyser les contraintes et à construire une première stratégie.

- **L'Orchestrator** : L'Orchestrator, ou orchestrateur, prend le relais lorsque les actions doivent être exécutées. Si le Brain définit ce qu'il faut faire, l'Orchestrator organise concrètement la manière de le faire. Il commence par structurer l'ordre des tâches. Il détermine quelle action doit être lancée en premier, laquelle doit suivre, et comment le processus doit évoluer. Dans un système agentique, cette organisation est essentielle, car une tâche complexe ne se réalise presque jamais en une seule étape.

- **Les Tools** : ou outils, donnent à l'agent la possibilité de dépasser le simple échange conversationnel. Grâce à eux, il ne se contente plus de produire des réponses : il peut interagir avec son environnement et exécuter des actions concrètes.

- **La Memory** : ou mémoire, permet à l'agent de garder le fil au fil des interactions. Sans mémoire, chaque échange serait traité comme un événement isolé. Avec elle, l'agent peut suivre une tâche dans le temps et conserver les informations importantes. On distingue généralement deux types de mémoire:

- La mémoire à court terme conserve le contexte immédiat : les derniers messages, les décisions récentes, les résultats des appels aux outils et l'état actuel de la tâche. Elle aide l'agent à rester cohérent pendant une session.
- La mémoire à long terme conserve des informations plus durables : les préférences de l'utilisateur, les objectifs récurrents, les décisions importantes ou certaines données issues d'interactions précédentes. Elle permet à l'agent de personnaliser davantage ses actions et d'éviter de redemander des informations déjà connues.

- **Le Supervisor** : Le Supervisor, ou superviseur, encadre le comportement de l'agent. Son rôle est indispensable, car un agent autonome ne doit pas pouvoir agir librement dans toutes les situations. Le superviseur intervient notamment à travers les mécanismes de validation humaine, souvent appelés Human-in-the-Loop. Certaines actions peuvent être préparées automatiquement, mais nécessitent une approbation avant d'être exécutées. Par exemple, un agent peut rédiger une offre ou préparer un e-mail, mais ne pas l'envoyer sans validation humaine.

2.2 Patrons architecturaux des systèmes agentiques

2.2.1 Quand utiliser des agents AI et LLM Workflows

La littérature distingue deux grandes familles d'architectures au sein des systèmes agentiques :

- **Workflows**: sont des systèmes où les grands modèles de langage (LLM) et les outils sont contrôlés par du code prédéfini.
- **AI agents**: Le terme "agent" a plusieurs définitions. Certains clients définissent un agent comme un système autonome qui fonctionne seul pendant longtemps et utilise différents outils

pour effectuer des tâches complexes. D'autres utilisent ce terme pour décrire des systèmes plus stricts qui suivent des processus prédéfinis. D'après Anthropic, nous considérons toutes ces variations comme des systèmes basés sur des agents, mais nous faisons une distinction importante entre les flux de travail et les agents.

—Lorsque vous développez des applications avec des LLM, nous recommandons de choisir la solution la plus simple possible et d'ajouter de la complexité uniquement si cela est nécessaire. Parfois, il n'est pas nécessaire de développer des systèmes basés sur des agents. Les systèmes multi-agents donnent souvent la priorité à l'efficacité des tâches plutôt qu'à la latence et au coût, il est donc important d'évaluer si ce compromis en vaut la peine [2].

—Lorsque la complexité est nécessaire, les flux de travail offrent prévisibilité et cohérence pour les tâches bien définies, tandis que les agents sont meilleurs lorsque la flexibilité et la prise de décision basée sur un modèle sont nécessaires à grande échelle. Cependant, pour de nombreuses applications, optimiser les appels LLM individuels avec récupération et exemples contextuels est souvent suffisant [2].

2.2.2 Modèles de conception d'agents d'IA et LLM Workflows

Dans les workflows, les LLM et les outils sont orchestrés via des **chemins de code prédéfinis**.

Le système suit une trajectoire prévisible et établie. Cinq patrons majeurs ont été identifiés :

Chaînage de prompts: Décomposition d'une tâche en séquence linéaire d'étapes, où la sortie d'un appel devient l'entrée du suivant. L'architecte échange intentionnellement la latence contre la précision en réduisant la portée de chaque appel.

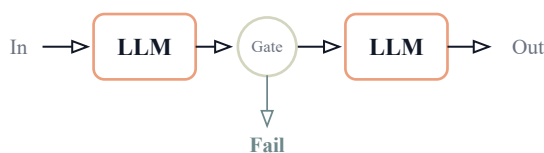


Figure 9: Chaînage de prompts avec étape de validation

Routage: Classification de l'entrée et orientation vers une tâche ou un modèle spécialisé. Ce patron garantit la séparation des préoccupations et permet d'utiliser des modèles de complexité différente selon les besoins.

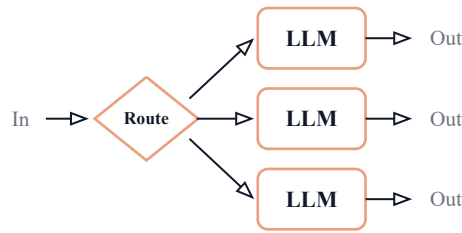


Figure 10: Workflow de routage vers des spécialistes LLM

Parallélisation: Traitement simultané de plusieurs aspects d'un problème, selon deux modèles : le *sectionnement* (chaque branche traite un aspect différent) et le *vote* (plusieurs branches traitent la même entrée pour fiabiliser le résultat).

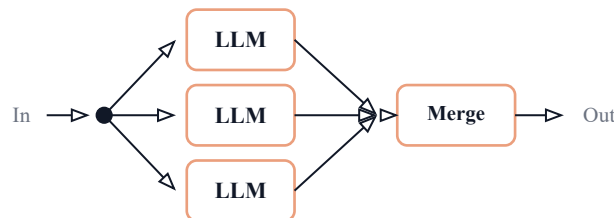


Figure 11: Workflow de parallélisation avec fusion des résultats

Orchestrateur-Travailleurs: Un LLM principal décompose dynamiquement une tâche complexe, délègue les sous-tâches à des LLM travailleurs et synthétise leurs résultats. Contrairement à la parallélisation, les sous-tâches ne sont pas prédéfinies — elles sont déterminées à la volée.

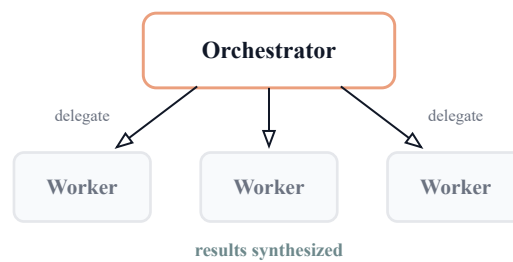


Figure 12: Workflow Orchestrateur-Workers

Évaluateur-Optimiseur: Création d'une boucle de rétroaction où un LLM génère un résultat et un autre le critique pour l'affiner. Ce patron convient lorsqu'il existe des critères d'évaluation clairs.

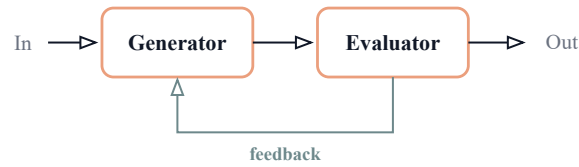


Figure 13: Workflow Evaluator–Optimizer

2.2.3 Agents autonomes

Là où les workflows sont des scripts, les agents autonomes sont des explorateurs. Ils opèrent via une boucle d’action-observation : à chaque étape, l’agent entreprend une action, observe le résultat de l’environnement (la *vérité terrain*), et ajuste son plan en conséquence.

Le cycle de vie d’un agent comprend : instruction → planification → action → feedback → itération → point de contrôle humain → terminaison. L’autonomie des agents implique cependant des coûts plus élevés et un risque d’erreurs composées, où une erreur précoce se propage à travers toute la chaîne [2].

2.2.4 Choix architectural pour ADAS-R2T

Pour notre projet, nous avons opté pour une approche hybride : un **flux de travail structuré** combinant quatre des cinq patrons (routage, parallélisation, orchestrateur-travailleurs, évaluateur-optimiseur). Ce choix offre la prévisibilité d’un workflow avec la puissance d’exécution d’un système agentique. Le chaînage simple a été écarté car le pipeline nécessite des branchements conditionnels et du parallélisme.

Patron	Application dans ADAS-R2T	Retenu
Chaînage	Pipeline partiellement linéaire (ingest → extract)	Partiel
Routage	Route chaque exigence vers les analyseurs pertinents	✓
Parallélisation	Analyseurs simultanés + N workers simultanés	✓
Orchestrateur-Workers	Coverage planner → workers de génération	✓
Évaluateur-Optimiseur	Evaluator → retry vers le planner	✓

Table 3: Correspondance entre les patrons architecturaux et leur usage dans ADAS-R2T.

2.3 LangGraph : framework d’orchestration

2.3.1 Pourquoi LangGraph

LangGraph est un framework d'orchestration conçu pour construire des workflows basés sur les grands modèles de langage (LLM) qui sont à la fois **stateful**, multi-étapes et orientés événements. Contrairement à une simple chaîne linéaire d'appels à un modèle de langage, LangGraph permet de représenter une application comme un graphe composé de nœuds, d'arêtes et de transitions conditionnelles.

Dans ce contexte, chaque nœud du graphe représente une étape de traitement, telle que l'analyse d'une exigence, l'extraction d'informations, la génération d'un cas de test, l'évaluation d'un résultat ou encore la correction d'une sortie. Les arêtes définissent les connexions entre ces étapes, tandis que la logique de transition permet de choisir dynamiquement le prochain traitement à exécuter.

LangGraph peut être considéré comme un moteur de workflow pour applications LLM. Il prend en charge des fonctionnalités essentielles pour les systèmes intelligents robustes, notamment la gestion de l'état, les branchements conditionnels, les boucles, les mécanismes de pause et de reprise, ainsi que la récupération après erreur. Ces caractéristiques sont particulièrement importantes dans un contexte industriel, où les résultats générés doivent être contrôlés, traçables et améliorables [1].

Dans le cadre du projet **ADAS-R2T**, LangGraph constitue un choix pertinent car le processus de génération des tests ADAS n'est pas strictement linéaire. Le pipeline doit être capable d'analyser les exigences, de générer des cas de test, de les évaluer, de détecter les incohérences, de demander une validation humaine et, si nécessaire, de régénérer uniquement les éléments rejetés. Cette logique correspond naturellement à une architecture sous forme de graphe.

2.3.2 LangGraph vs LangChain ?

LangChain reste adapté aux workflows simples et linéaires, par exemple lorsqu'il s'agit d'enchaîner quelques appels successifs à un modèle de langage, de construire un système de résumé ou de mettre en place un mécanisme de recherche documentaire basique. Dans ce type de cas, le traitement suit généralement une séquence fixe et prévisible [3].

En revanche, LangGraph devient plus approprié lorsque le cas d'usage implique des workflows complexes et non linéaires. Il est particulièrement utile lorsque l'application nécessite :

- des chemins conditionnels selon la qualité ou le type des données traitées ;
- des boucles d'amélioration ou de régénération ;
- des étapes de validation humaine (**Human-in-the-Loop**) ;
- une coordination entre plusieurs agents spécialisés ;
- une exécution asynchrone ou orientée événements ;
- une gestion persistante de l'état du workflow.

Dans notre projet, ces besoins sont présents à plusieurs niveaux. Par exemple, un cas de test rejeté par l'utilisateur ne doit pas entraîner la relance complète du pipeline. Il doit uniquement déclencher une branche de correction ou de régénération ciblée. De même, l'évaluation automatique des cas générés peut conduire à différentes décisions : validation, correction, rejet, ou demande d'intervention humaine.

2.3.3 Complémentarité entre LangGraph et LangChain

LangGraph ne remplace pas LangChain. Il s'appuie au contraire sur ses composants pour construire les différentes étapes du workflow. LangChain fournit les briques de base nécessaires à l'interaction avec les modèles de langage et les ressources externes, tandis que LangGraph organise ces briques dans un graphe d'exécution contrôlé.

Ainsi, dans une architecture basée sur LangGraph, il reste possible d'utiliser des composants LangChain tels que :

- ChatOpenAI ou d'autres connecteurs vers des modèles de langage ;
- PromptTemplate pour structurer les instructions envoyées au modèle ;
- les **retrievers** pour récupérer des informations pertinentes ;
- les **document loaders** pour charger des documents ou fichiers d'entrée ;
- les **tools** pour connecter le système à des services externes ou à des fonctions spécifiques.

De ce fait, LangChain peut être vu comme une bibliothèque de composants, tandis que LangGraph joue le rôle d’un orchestrateur de workflow. Cette complémentarité permet de concevoir une architecture plus robuste, modulaire et adaptée aux exigences du projet **ADAS-R2T**.

① Note

Dans ADAS-R2T, LangGraph est utilisé comme couche d’orchestration principale. Il permet de coordonner les agents spécialisés, de gérer l’état du pipeline, d’intégrer la validation humaine et de contrôler les boucles de correction ou de régénération des cas de test.

2.3.4 Concepts fondamentaux

Un graphe LangGraph se compose de trois éléments : les **nœuds** (fonctions async qui lisent et écrivent dans l’état partagé), les **arêtes** (transitions entre nœuds, linéaires ou conditionnelles), l’état (un TypedDict partagé qui accumule les résultats au fil de l’exécution).

Le concept de *reducer* est particulièrement important pour les systèmes parallèles : lorsque plusieurs nœuds écrivent simultanément dans le même champ (par exemple, plusieurs workers ajoutant des cas de test), le reducer (typiquement `operator.add` pour les listes) fusionne les résultats sans conflit.

2.4 Ingénierie des prompts et du contexte

2.4.1 Du prompt engineering au context engineering

L’ingénierie des prompts traditionnelle consiste à formuler des instructions claires pour le LLM. L’ingénierie du contexte va plus loin : elle conçoit l’*environnement complet* dans lequel le LLM opère, connaissances, contraintes, format de sortie [4].

Pour ce projet, nous avons adopté le framework (**MISBAH**), un cadre méthodologique en cinq étapes pour la construction de contextes LLM de haute qualité :

1. **Le Principe** — alignement de l’intention : définir l’objectif stratégique unique que le modèle doit poursuivre, élevant la qualité de « statistiquement probable » à « stratégiquement ciblé ».

2. **La Formulation** — amorçage comportemental (*behavioral priming*) : activer les réseaux neuronaux du modèle associés à un profil d'expert spécifique, reproduisant non seulement ses connaissances mais son mode de raisonnement.
3. **Le Protocole** — raisonnement structuré (*chain-of-thought*) : imposer des étapes analytiques strictes et séquentielles pour réduire les erreurs logiques et améliorer la cohérence.
4. **Les Standards** — guidage négatif : spécifier explicitement les comportements interdits, éliminant des catégories entières de sorties faibles.
5. **Le Résultat** — format de sortie : définir la structure exacte attendue pour garantir des résultats cohérents et exploitables par le code.

L'application de ce framework aux prompts du projet a démontré des améliorations significatives, notamment la réduction des descriptions vides de 52% à 0% et l'élimination complète des erreurs de formatage.

2.5 Travaux connexes

La génération automatique de scénarios et de cas de test pour les systèmes ADAS/ADS constitue aujourd'hui un axe de recherche important, en raison de la complexité croissante des fonctions de conduite automatisée et de la difficulté de couvrir l'ensemble des situations de conduite possibles. Les travaux existants peuvent être regroupés en plusieurs familles : les approches fondées sur les scénarios et les critères de couverture, les méthodes exploitant les descriptions textuelles, les approches basées sur les grands modèles de langage, ainsi que les solutions industrielles alignées avec les standards ASAM.

2.5.1 Validation ADAS/ADS et génération de scénarios conformes à SOTIF

La validation des systèmes avancés d'aide à la conduite et des systèmes de conduite automatisée repose de plus en plus sur le **scenario-based testing**. Cette approche consiste à évaluer le comportement du système sous test dans des situations représentatives de l'environnement réel, incluant l'infrastructure routière, les acteurs dynamiques, les conditions météorologiques et les interactions entre véhicules.

Dans ce contexte, la norme **SOTIF (Safety of the Intended Functionality, ISO 21448)** souligne l'importance de générer des suites de scénarios capables de couvrir l'espace opéra-

tionnel du système tout en identifiant les situations potentiellement dangereuses. Cependant, plusieurs travaux montrent que la norme ne définit pas précisément comment sélectionner les scénarios, comment mesurer leur couverture, ni comment évaluer leur capacité à révéler des défaillances. Cette limite rend son application pratique difficile dans des environnements industriels complexes.

Des travaux récents proposent donc d’utiliser des modèles de variabilité, des stratégies de sampling et des critères de couverture pour représenter l’espace des scénarios et générer des suites de tests plus efficaces. Par exemple, les approches combinatoires permettent de couvrir systématiquement différentes interactions entre entités de scénario, tandis que les stratégies de **selective sampling** tiennent compte de la complexité des scénarios. L’évaluation par **mutation testing** permet ensuite d’estimer la capacité d’une suite de scénarios à détecter des fautes potentielles dans le système sous test [5].

Ces approches sont particulièrement intéressantes pour notre projet, car elles mettent en évidence deux exigences essentielles : la couverture des scénarios possibles et la capacité à détecter des comportements dangereux. Toutefois, elles se concentrent principalement sur la génération de scénarios à partir d’espaces formalisés ou de paramètres de simulation, alors que notre projet part d’un autre type d’entrée : les exigences fonctionnelles ADAS rédigées en langage naturel.

2.5.2 Génération de scénarios à partir de descriptions textuelles

Une autre famille de travaux s’intéresse à la génération de scénarios de test à partir de descriptions textuelles, notamment des rapports d’accidents, des récits de trafic ou des descriptions formulées par des experts. Cette orientation est particulièrement pertinente, car les textes décrivent souvent des relations causales, des interactions dynamiques et des contextes de conduite qui ne sont pas toujours faciles à extraire à partir de données visuelles.

Le système **Txt2Sce** illustre cette tendance. Il exploite des rapports d’accidents impliquant des véhicules autonomes afin de générer des fichiers **OpenSCENARIO**. Le processus consiste d’abord à extraire les éléments clés du rapport textuel à l’aide d’un LLM, puis à produire un scénario initial conforme au standard OpenSCENARIO. Ensuite, le système applique des opérations de désassemblage, de mutation et de réassemblage afin de construire des arbres de scénarios plus diversifiés. Les auteurs montrent que cette méthode permet de produire plusieurs

milliers de scénarios valides et de détecter différents comportements inattendus d'Autoware, notamment en matière de sécurité, de fluidité et de prise de décision [6].

Cette approche présente plusieurs avantages : elle exploite des descriptions issues du monde réel, produit des fichiers standards réutilisables, et permet d'analyser les conditions qui déclenchent des comportements inattendus. Toutefois, elle se focalise principalement sur les rapports d'accidents et la génération de scénarios de simulation. Dans notre cas, l'objectif est différent : il s'agit de transformer des exigences fonctionnelles ADAS en plans et cas de test structurés, tout en garantissant la traçabilité entre chaque exigence et les tests générés.

2.5.3 Apport des LLMs dans la génération de scénarios de test

Les grands modèles de langage ouvrent de nouvelles perspectives pour l'automatisation des tâches liées à l'ingénierie des exigences et à la génération de scénarios. Leur capacité à comprendre le langage naturel, à extraire des entités, à structurer des informations et à produire des sorties semi-formalisées les rend particulièrement adaptés aux domaines où les spécifications sont majoritairement textuelles.

Le framework **Text2Scenario** propose une approche où un LLM agit comme parser de descriptions textuelles de scénarios. Le système s'appuie sur un référentiel hiérarchique de composants de scénario, comprenant notamment la topologie de la route, les infrastructures, les changements temporaires, les participants au trafic, le climat et le véhicule ego. À partir d'une description textuelle, le LLM sélectionne les éléments correspondants, puis un générateur basé sur DSL assemble ces éléments pour produire des fichiers exécutables dans un simulateur. Les auteurs mettent également en avant un pipeline de **prompt engineering** comprenant le **role setting**, le **few-shot learning**, le **chain-of-thought**, la vérification syntaxique et la **self-consistency** [7].

Les résultats de cette approche montrent que l'utilisation d'un LLM avancé, notamment GPT-4, améliore fortement la qualité de l'extraction des éléments de scénario et réduit le temps de construction par rapport à des experts humains. Néanmoins, la génération reste limitée par la robustesse du LLM, la dépendance à un référentiel d'éléments préexistant et la nécessité de vérifier la faisabilité des fichiers produits.

Ces constats sont directement liés à notre projet. En effet, ADAS-R2T ne doit pas se contenter d'un simple appel à un LLM. Le système doit intégrer des mécanismes de contrôle, de

validation, de correction et de supervision humaine afin de réduire les risques d’hallucination, d’incohérence ou de génération hors périmètre.

2.5.4 Exploitation des sources ouvertes et OSINT pour les scénarios ADAS

Certains travaux explorent également l’utilisation de sources ouvertes, telles que les données publiques disponibles sur Internet, pour enrichir la génération de scénarios AD/ADAS. L’approche OSINT combinée aux LLMs consiste à collecter des descriptions d’incidents de trafic à l’aide de techniques de web scraping, puis à structurer ces informations dans un format compatible avec un langage de description de scénarios tel qu’OpenSCENARIO.

Une étude récente propose un pipeline combinant web scraping, LLMs et OpenSCENARIO XML v1.2.0. Le système identifie des sources publiques, extrait des informations qualitatives sur les incidents de trafic, puis utilise des LLMs pour structurer et compléter les scénarios. L’étude distingue deux niveaux de validation : une validation syntaxique basée sur le schéma XSD d’OpenSCENARIO et une validation sémantique portant sur la cohérence, la complétude, la clarté et la conformité du scénario généré à la description originale [8].

Cette approche est intéressante car elle montre que les données ouvertes peuvent constituer une source complémentaire pour identifier des scénarios réalistes et potentiellement rares. Cependant, elle présente plusieurs limites : l’accès aux sources est contraint par des considérations légales et éthiques, la qualité des textes disponibles varie fortement, et la génération par LLM nécessite une validation rigoureuse pour éviter les hallucinations et les incohérences.

Dans ADAS-R2T, l’intégration de vidéos de conduite et de feedback utilisateur peut jouer un rôle similaire à celui des sources ouvertes : enrichir la base des scénarios et rapprocher les tests générés des situations réelles, tout en maintenant un cadre contrôlé et traçable.

2.5.5 Standards ASAM et interopérabilité des scénarios

Les standards ASAM jouent un rôle important dans la génération, la gestion et l’exécution des scénarios ADAS/ADS. **OpenSCENARIO** permet de décrire les scénarios dynamiques, **OpenDRIVE** fournit une représentation standardisée du réseau routier, tandis qu’**OpenLABEL** facilite l’organisation et l’annotation des données de scénarios.

Une application industrielle présentée par ASAM et IAV montre l'intérêt d'une génération de scénarios assistée par IA. Dans cette approche, des LLMs sont utilisés pour transformer des spécifications de tests, des rapports d'accidents, des exigences internes ou des connaissances d'experts en scénarios compatibles avec les chaînes d'outils de simulation. Le système s'appuie également sur des étapes de prétraitement et de post-traitement afin d'améliorer la qualité, la modifiabilité et la paramétrisation des scénarios générés [9].

Cette approche industrielle confirme plusieurs éléments importants pour notre projet : premièrement, l'utilisation d'un LLM doit être intégrée dans un pipeline contrôlé ; deuxièmement, les standards ASAM facilitent l'interopérabilité entre les outils ; troisièmement, la gestion des scénarios ne se limite pas à leur génération, mais inclut leur organisation, leur paramétrage, leur réutilisation et leur analyse.

Bien que notre projet ne vise pas directement la génération de fichiers OpenSCENARIO dans sa première version, l'orientation vers des formats standards constitue une perspective importante. À terme, ADAS-R2T pourrait exploiter les cas de test générés pour produire des scénarios compatibles avec des environnements de simulation tels que CARLA ou des toolchains basées sur les standards ASAM.

2.5.6 Analyse comparative des travaux étudiés

Les travaux étudiés présentent des contributions complémentaires. Les approches fondées sur SOTIF et le sampling se concentrent sur la couverture et l'évaluation des suites de scénarios. Les approches comme Txt2Sce et Text2Scenario exploitent les LLMs pour transformer des descriptions textuelles en scénarios exécutables. Les travaux basés sur l'OSINT montrent l'intérêt des sources ouvertes pour extraire des situations réalistes. Enfin, l'approche ASAM/IAV met en évidence l'importance d'un pipeline industriel standardisé, compatible avec les formats ASAM.

Travail	Objectif principal	Méthode	Limites / différence avec ADAS-R2T
Birkemeyer et al.	Générer des suites de scénarios conformes à SOTIF	Feature models, sampling, mutation testing	Ne traite pas directement les exigences fonctionnelles textuelles ni la génération de cas de test traçables.
Txt2Sce	Générer des scénarios à partir de rapports d’accidents	LLM, OpenSCENARIO, mutation, scenario tree	Se concentre sur les rapports d’accidents et les scénarios de simulation plutôt que sur les exigences ADAS.
OSINT + LLM	Exploiter des sources ouvertes pour construire des scénarios AD/ADAS	Web scraping, LLM, OpenSCENARIO XML	Dépend fortement de la disponibilité et de la qualité des sources ouvertes.
Text2Scenario	Convertir des descriptions textuelles en scénarios exécutables	LLM parser, référentiel hiérarchique, DSL, CARLA	Nécessite un référentiel de composants et une validation de faisabilité des scénarios.
ASAM / IAV	Industrialiser la génération et la gestion de scénarios ADAS	LLM, preprocessing, postprocessing, standards ASAM	Approche orientée scénario ; ADAS-R2T se concentre d’abord sur requirements-to-tests.

Table 4: Comparaison synthétique des travaux connexes liés à la génération de scénarios et de tests ADAS

2.5.7 Positionnement du projet ADAS-R2T

Au regard des travaux précédents, le projet **ADAS-R2T** se positionne à l’intersection de trois axes : l’ingénierie des exigences, la génération automatique de tests et l’orchestration agentique des workflows LLM.

Contrairement aux approches centrées sur la génération de scénarios de simulation à partir de rapports d’accidents ou de descriptions textuelles, notre projet prend comme entrée principale des exigences fonctionnelles ADAS. L’objectif n’est pas uniquement de produire un scénario de conduite, mais de générer des plans de test et des cas de test exploitables par les ingénieurs de validation.

De plus, ADAS-R2T se distingue par l’intégration d’une architecture multi-agents orchestrée par LangGraph. Cette architecture permet de répartir le processus entre plusieurs agents spécialisés : analyse des exigences, extraction des dimensions fonctionnelles, génération des cas de test, évaluation automatique, correction, mémorisation des feedbacks et validation humaine.

Le projet se distingue également par l’importance accordée à la traçabilité. Chaque cas de test généré doit rester lié à son exigence source, ce qui facilite l’audit, la maintenance, la couverture fonctionnelle et l’évolution des tests lors des changements de spécifications. Cette dimension est

essentielle dans un contexte automobile où les exigences de sécurité, de qualité et de conformité sont particulièrement strictes.

Enfin, l'intégration du principe **Human-in-the-Loop** permet de conserver l'expert humain au centre du processus. Les résultats générés par l'IA ne sont pas acceptés automatiquement : ils peuvent être validés, rejetés, supprimés ou régénérés en fonction du feedback de l'utilisateur. Ce mécanisme répond aux limites identifiées dans les travaux existants concernant la fiabilité des LLMs, les hallucinations et la nécessité d'un contrôle métier.

Ainsi, ADAS-R2T ne remplace pas les approches existantes de génération de scénarios, mais les complète. Il propose une couche intelligente de transformation des exigences vers des tests, avec une perspective d'évolution vers l'intégration de formats standards tels qu'OpenSCENARIO et vers l'exploitation de données réelles issues de vidéos de conduite.

Architecture Générale du Projet

Ce chapitre présente l'architecture du système ADAS-R2T selon une approche descendante : nous partons d'une vue macroscopique du pipeline, puis nous détaillons progressivement chaque couche technique jusqu'aux mécanismes internes du graphe agentique.

3.1 Vue d'ensemble

Le système **ADAS-R2T** repose sur un principe simple : transformer des **entrées métier** (exigences fonctionnelles, vidéos de conduite) en **sorties exploitables** (cas de test structurés, scénarios de validation). Entre ces deux extrémités, un “**pipeline intelligent**” orchestre le travail de dix-neuf nœuds spécialisés.

3.1.1 Flux global

À son niveau le plus abstrait, le système fonctionne comme une chaîne de transformation en trois temps :

- **Entrée** : l'utilisateur fournit un fichier Excel contenant les exigences fonctionnelles ADAS (et éventuellement une vidéo de conduite réelle).
- **Traitement** : le “pipeline” agentique analyse, planifie, génère et évalue les cas de test de manière autonome.
- **Sortie** : un fichier Excel structuré contenant les cas de test prêts à être exécutés par l'équipe de validation.

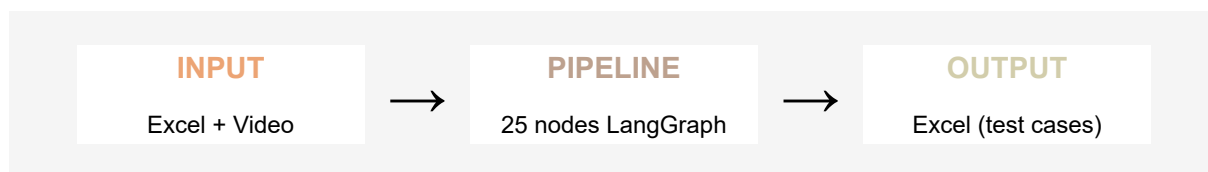


Figure 14: Vue synthétique du pipeline ADAS-R2T

Ce flux, en apparence linéaire, cache en réalité une mécanique bien plus riche. Le “pipeline” ne se contente pas de « traduire » des exigences en tests : il analyse chaque exigence sous plusieurs angles, planifie la couverture de test, génère les cas en parallèle, les évalue automatiquement, puis les soumet à l'utilisateur pour validation avant de produire le livrable final.

3.1.2 Les quatre étapes du “pipeline”

Le traitement interne se décompose en quatre grandes étapes, chacune correspondant à un ensemble de nœuds dans le graphe agentique :

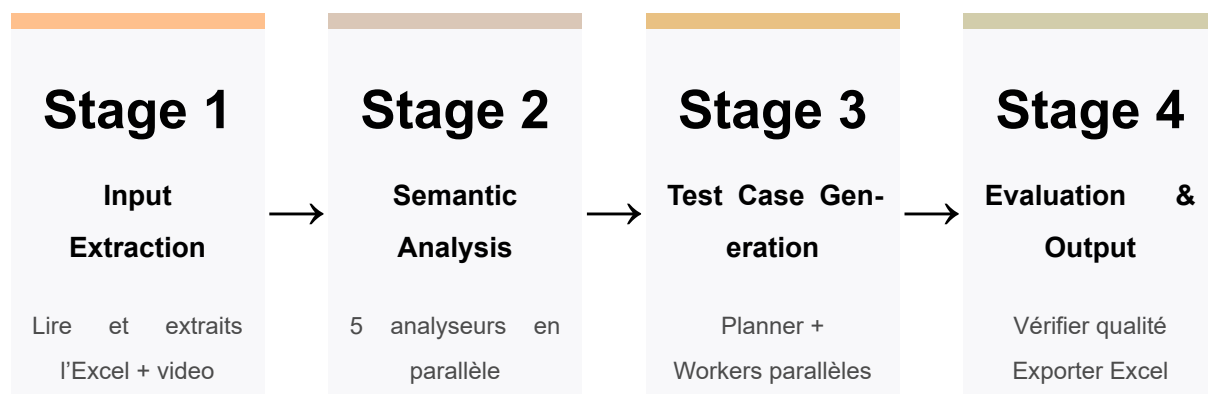


Figure 15: Les quatre étapes du pipeline de generation.

Étape 1 : Extraction des entrées. Le système ingère le fichier Excel, identifie la structure du document (en-têtes, colonnes, “flow table”), et extrait les exigences fonctionnelles sous une forme structurée exploitable par les étapes suivantes.

Étape 2 : Analyse sémantique. Chaque exigence est soumise à cinq analyseurs spécialisés fonctionnant en parallèle : transitions d'états, contraintes temporelles, interactions homme-machine, logique de calcul et analyse générique. Cette analyse multidimensionnelle permet de capturer la richesse sémantique de chaque exigence.

Étape 3 : Génération des cas de test. Un planificateur détermine la stratégie de couverture pour chaque exigence (cas nominaux, limites, négatifs, rares), puis des “workers” parallèles génèrent les cas de test correspondants. Un synthétiseur élimine ensuite les doublons et consolide les résultats.

Étape 4 : Évaluation et sortie. Chaque cas de test généré est évalué automatiquement (détection de contradictions, vérification du périmètre, cohérence des valeurs). Les résultats sont ensuite soumis à une revue humaine avant “HITL” d’être exportés au format Excel.

3.1.3 Les trois modes d’entrée

Le système supporte trois modes de fonctionnement, offrant une flexibilité adaptée à différents contextes d’utilisation :

Mode	Entrées	Sorties
Excel seul	Fichier d’exigences fonctionnelles	Cas de test structurés
Vidéo seule	Vidéo de conduite réelle	Scénarios de test avec raisonnement causal
Excel + Vidéo	Exigences + vidéo	Cas de test enrichis par les scénarios vidéo

Table 5: Modes d’entrée et sorties générées par “ADAS-R2T”

- Le mode *Excel seul* constitue le cas d’usage principal : l’équipe de validation dispose d’un document d’exigences et souhaite générer les cas de test correspondants.
- Le mode *Vidéo seule* permet d’exploiter des vidéos de conduite réelle pour en extraire des scénarios de test basés sur un raisonnement causal (cause, effet, conséquence).
- Le mode *Excel + Vidéo* combine les deux approches : les cas de test générés à partir des exigences sont enrichis par les observations extraites de la vidéo.

Cette distinction entre modes se matérialise au niveau du graphe par un routage conditionnel dès le premier nœud, orientant le flux de traitement vers les branches appropriées.

3.2 Architecture technique globale

Le système “ADAS-R2T” s’articule autour d’un noyau central le “*pipeline*” d’orchestration qui coordonne l’ensemble des composants techniques. Chaque composant remplit un rôle précis et communique avec le “pipeline” via des interfaces bien définies.

3.2.1 Vue des composants

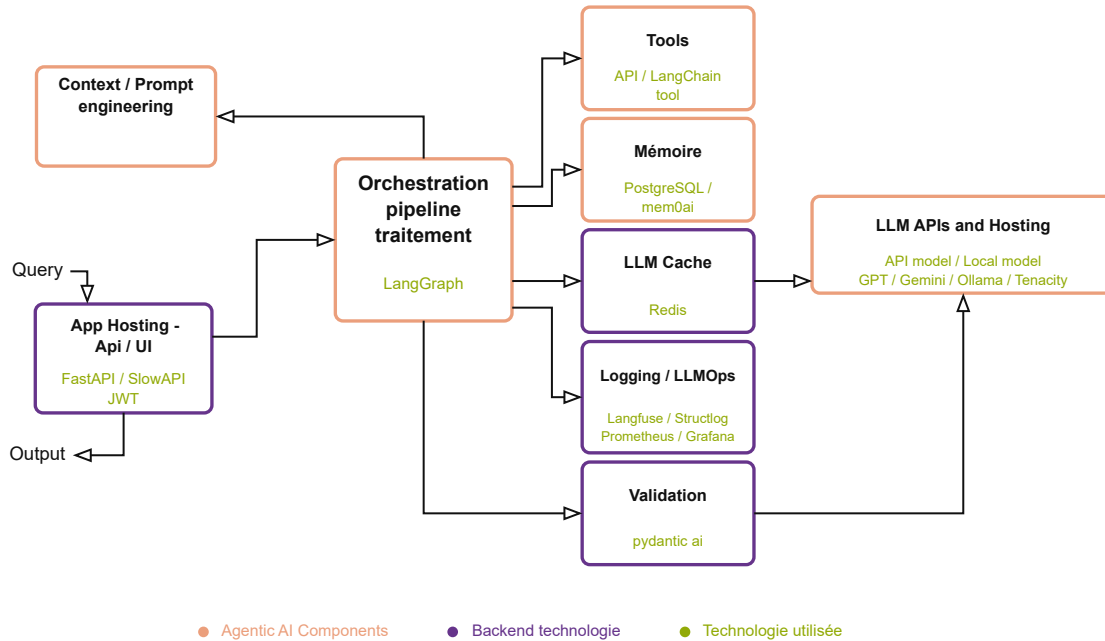


Figure 16: Architecture globale des composants agentiques et techniques d'ADAS-R2T

Le schéma ci-dessus fait apparaître huit composants principaux, organisés autour du “pipeline” d’orchestration. Nous les décrivons ci-dessous en suivant le flux d’une requête typique.

3.2.2 “Pipeline” d’orchestration "LangGraph"

Le cœur du système est un graphe d’exécution construit avec “LangGraph”, le framework d’orchestration d’agents de “LangChain”. Ce graphe définit l’enchaînement des dix-neuf nœuds de traitement, gère le parallélisme et assure la persistance de l’état entre les étapes. C’est lui qui décide quel nœud s’exécute, dans quel ordre, et comment les résultats circulent d’un agent à l’autre.

Le choix de “LangGraph”, plutôt qu’un enchaînement séquentiel de prompts, permet de bénéficier de mécanismes avancés : exécution parallèle via "Send()", interruptions pour la revue humaine, retour arrière "Time Travel" sans perte de contexte, et reprise automatique après une panne.

3.2.3 Modèles de langage

Lors de la conception du système, une question pratique importante s'est posée : et si demain nous souhaitions utiliser un modèle de langage d'une autre entreprise que celle de départ ? Devrions-nous réécrire de larges pans du code ? C'était un véritable problème, car chaque fournisseur de services avait sa propre méthode de communication.

Le problème : traiter séparément avec chaque fournisseur aurait rendu le code complexe et difficile à maintenir. Chaque simple modification aurait nécessité des changements à plusieurs endroits, un véritable cauchemar pour tout programmeur.

La solution astucieuse : la solution a consisté à implémenter une astuce de programmation élégante appelée "Factory Pattern". L'idée est simple et efficace : au lieu de communiquer directement avec "OpenAI" ou "Cohere", les composants du système communiquent avec un seul intermédiaire, "LLMProviderFactory". Cet intermédiaire est seul responsable de la communication avec chaque fournisseur. Grâce à un simple paramètre dans le fichier de configuration, cette « fabrique » génère l'expert approprié et le met à contribution.

L'avantage immédiat : changer de modèle d'IA est devenu aussi simple que de changer de chaîne de télévision. Si nous souhaitons ajouter un nouveau modèle ultérieurement, il nous suffit d'apprendre à l'« usine » comment communiquer avec lui, sans toucher au reste du système. C'est une solution simple et claire qui rend le système robuste et facile à développer.

Cette dualité n'est pas figée : chaque nœud du "pipeline" peut être configuré indépendamment pour utiliser l'un ou l'autre fournisseur, via les variables d'environnement. Cette flexibilité permet d'adapter le choix du modèle au rapport coût-performance de chaque tâche. Le mécanisme de "retry" avec "backoff" exponentiel (via "Tenacity") assure la robustesse face aux erreurs transitoires des API externes.

3.2.4 Ingénierie des prompts

Les instructions envoyées aux modèles de langage ne sont pas codées en dur dans le code source. Chaque nœud charge son "prompt" depuis un fichier "Markdown" dédié, stocké dans un répertoire centralisé. Cette séparation entre logique de traitement et contenu des prompts facilite l'itération rapide : un ingénieur peut modifier un "prompt" sans toucher au code "Python", et chaque modification est traçable dans l'historique "Git".

Les prompts s'appuient sur les principes du framework "MISBAH", qui structure les instructions en sections claires : contexte métier, format de sortie attendu, exemples et contraintes à respecter.

3.2.5 Mémoire et persistance ("PostgreSQL")

Le système exploite "PostgreSQL" pour deux fonctions distinctes de mémoire :

La **mémoire de session** (courte durée) est assurée par le "checkpoint" de "LangGraph". À chaque étape du "pipeline", l'état complet est sauvegardé dans "PostgreSQL" sous forme de "checkpoint" chiffré "AES". Ce mécanisme rend possible l'interruption pour revue humaine, le retour arrière ("Time Travel"), et la reprise après panne sans perte de travail.

La **mémoire à long terme** "cross-session" stocke les connaissances acquises au fil des utilisations. Elle se décline en trois portées : sémantique applicative règles partagées par tous les utilisateurs, sémantique utilisateur préférences individuelles, et épisodique historique des revues. La recherche dans cette mémoire s'appuie sur des "embeddings" vectoriels "pgvector" pour retrouver les connaissances pertinentes par similarité sémantique.

3.2.6 Observabilité ("Langfuse", "Prometheus", "Grafana")

L'observabilité du système repose sur trois piliers complémentaires :

- **"Langfuse"** capture chaque appel "LLM" avec ses paramètres, sa durée, ses tokens consommés et sa réponse. Ce traçage fin permet d'identifier les prompts sous-performants, de mesurer les coûts, et de déboguer les cas de génération insatisfaisants.
- **"Prometheus"** collecte les métriques opérationnelles du système : nombre de "pipelines" exécutés, durée par nœud, taux d'erreur, décisions "HITL", et opérations mémoire. Ces métriques sont exposées via un "endpoint" "/metrics" au format standard.
- **"Grafana"** consomme les métriques de "Prometheus" et les présente sous forme de tableaux de bord visuels. Un "dashboard" dédié "ADAS-R2T Pipeline Monitor" offre une vue en temps réel sur la santé et les performances du système.

Le logging structuré, assuré par "structlog", complète ce dispositif en produisant des logs au format "JSON" exploitables par des outils d'analyse.

3.2.7 Validation et évaluation

Le module d'évaluation constitue le gardien de la qualité du système.

3.2.8 Choix techniques et justifications

Composant	Technologie	Justification
Orchestration	LangGraph	Graphe d'agents avec parallélisme, interruptions et persistance native
API	FastAPI	Performance async, documentation automatique, écosystème Python
Base de données	PostgreSQL pgvector	Robustesse, support vectoriel pour la recherche sémantique
Monitoring	Prometheus "Grafana"	Standard industriel, dashboards personnalisables
LLM tracing	Langfuse	Spécialisé LLMOps, open source, intégré à LangChain
Logging	structlog	Logs structurés JSON, décorateurs de nœud
Conteneurisation	Docker Compose	Déploiement reproductible, isolation des services
Sécurité	JWT + API Key + AES	Authentification double, chiffrement au repos

Table 6: Choix technologiques retenus pour l'architecture ADAS-R2T

3.3 Architecture du graphe agentique

Le "pipeline" de traitement constitue le cœur technique du système. Il prend la forme d'un graphe orienté, construit avec "LangGraph", où chaque nœud représente un agent spécialisé dans une tâche précise. Ce graphe ne se parcourt pas de manière linéaire : selon le mode d'entrée choisi, certaines branches s'activent et d'autres sont ignorées. Des mécanismes de parallélisme, de boucle et d'interruption viennent enrichir ce parcours.

3.3.1 Vue d'ensemble du graphe

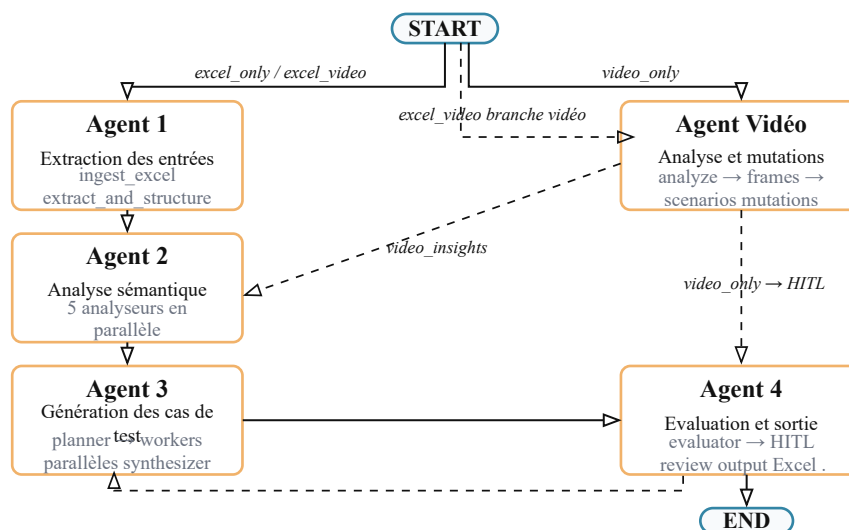


Figure 17: Vue globale du pipeline

Le graphe opère à l'intérieur de trois niveaux d'encapsulation, visibles sur la figure ci-dessus :

- Le **scope session** englobe l'exécution d'un "pipeline" unique. C'est à ce niveau que le "checkpoint" sauvegarde l'état à chaque étape, rendant possibles l'interruption et la reprise.
- Le **scope utilisateur** regroupe l'ensemble des sessions d'un même utilisateur. La mémoire sémantique et épisodique de l'utilisateur persiste à ce niveau.
- Le **scope application** couvre l'ensemble du système. Les règles de qualité apprises et partagées par tous les utilisateurs sont stockées à ce niveau.

3.3.2 Agent 1 : Extraction des entrées

La première étape gère l'ingestion des fichiers fournis par l'utilisateur. Selon le mode d'entrée, le graphe active l'une ou plusieurs des branches suivantes :

- **ingest excel** : Ce nœud prend en charge la lecture du fichier Excel. Il identifie la structure du document (en-têtes, colonnes des données, "flow table"), extrait un aperçu des premières lignes, et prépare les données brutes pour l'étape suivante. Ce nœud ne fait pas appel au LLM : son traitement est entièrement déterministe, basé sur la bibliothèque openpyxl.
- **extract and structure** : À partir des données brutes, ce nœud fait appel au LLM pour transformer le contenu des cellules en exigences structurées. Chaque exigence se voit attribuer un identifiant unique, un texte normalisé, et des métadonnées (variables, conditions, seuils). C'est ici que le passage du langage naturel à une représentation exploitable s'opère.

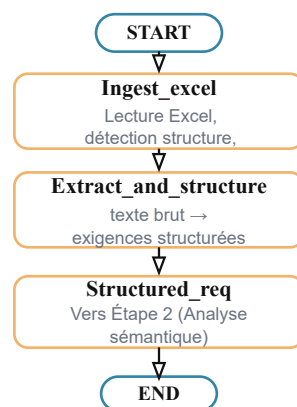


Figure 18: Agent 1 Workflows

3.3.3 Agent vidéo : analyse et mutations

Lorsque l'utilisateur fournit une vidéo de conduite, une branche parallèle s'active. Elle se compose de quatre nœuds :

- **analyze video** : Ce nœud extrait les frames clés de la vidéo à intervalles réguliers, puis applique un algorithme de détection de changement de scène (basé sur la différence de pixels entre frames consécutives) pour ne retenir que les moments significatifs. Le résultat est un ensemble de frames clés accompagnées de leurs timestamps.
- **video frame analyzer** : Chaque frame clé est analysée individuellement par un LLM multimodal . L'analyse produit pour chaque frame : une description de la scène, la vitesse estimée du véhicule ego, les objets détectés (véhicules, piétons, panneaux), les conditions environnementales, et l'action en cours du véhicule. Les frames sont analysées en parallèle grâce à un semaphore qui contrôle la concurrence.
- **video scenario builder** : À partir des analyses de frames, ce nœud reconstruit des scénarios complets en établissant des chaînes causales. Chaque scénario se structure en trois temps : la cause (ce qui déclenche la situation), l'effet (la réaction immédiate), et la conséquence (l'impact sur la sécurité). Cette approche, inspirée de la méthode "Txt2Sce", donne aux scénarios une profondeur que ne permettrait pas une simple description factuelle.
- **video scenario mutator** : Le dernier nœud de la branche vidéo génère des variations réalistes à partir de chaque scénario de base. Cinq stratégies de mutation sont appliquées : variation de la cause, variation de l'effet, augmentation de la complexité, changement d'environnement, et inversion des rôles. Ce processus produit entre quinze et vingt-cinq scénarios dérivés pour chaque scénario source, couvrant ainsi un large spectre de situations de conduite.

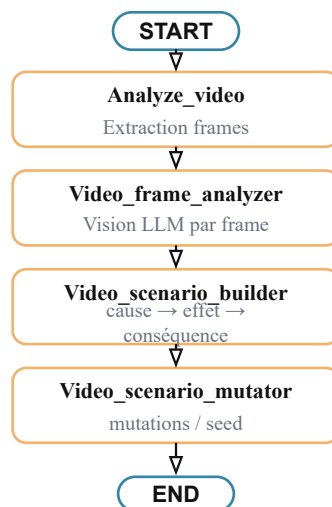


Figure 19: Agent video Workflows

3.3.4 Agent 2 : Analyse sémantique

Une fois les exigences structurées, chaque exigence est soumise à un ensemble d'analyseurs spécialisés. Le nœud "route_requirement" examine le contenu de l'exigence et l'oriente vers les analyseurs pertinents.

Cinq analyseurs fonctionnent en parallèle :

- **state analyzer** : Identifie les transitions d'états décrites dans l'exigence (par exemple : "ACC" passe de "Off" à "Active" lorsque le bouton est pressé). Il extrait les états initiaux, les événements déclencheurs, et les états finaux.
- **timing analyzer** : Détecte les contraintes temporelles (délais, durées, "timeouts") et les traduit en conditions de test vérifiables (par exemple : « l'activation doit se produire en moins de 500 ms »).
- **hmi analyzer** : Repère les interactions homme-machine : boutons, affichages, alertes sonores, témoins lumineux. Il identifie les entrées utilisateur et les retours attendus de l'interface.
- **computation analyzer** : Extrait la logique de calcul et les formules.
- **generic analyzer** : Capture les aspects qui n'entrent dans aucune des catégories précédentes : conditions environnementales, contraintes de périmètre, cas aux limites.
- **merge analyses** : Consolide les résultats de tous les analyseurs en une synthèse unique par exigence, créant ainsi un contexte riche pour la génération des cas de test.

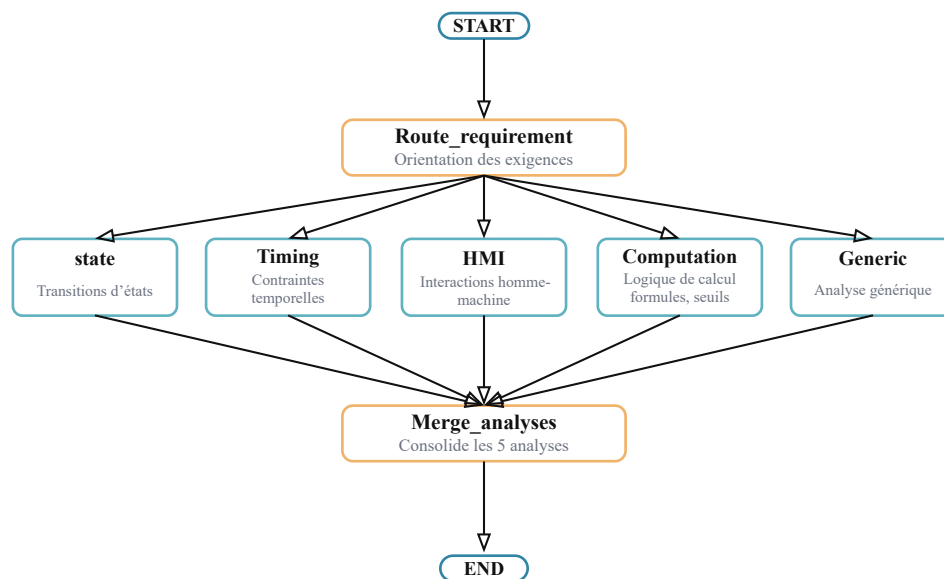


Figure 20: Agent 3 Analyse sémantique

3.3.5 Agent 3 : Génération des cas de test

La génération se décompose en quatre nœuds qui opèrent selon un schéma planificateur-workers :

- **coverage planner** : Ce nœud déterministe (sans appel "LLM") élabore la stratégie de couverture pour chaque exigence. Il détermine combien de cas de test sont nécessaires et de quel type : nominaux (fonctionnement normal), aux limites (valeurs seuils), négatifs (conditions de défaillance), et rares (combinaisons inhabituelles). Ce planificateur s'appuie sur la richesse de l'analyse sémantique pour ne rien laisser de côté.
- **plan single req** : Pour chaque exigence, ce nœud génère les blueprints (plans détaillés) des cas de test via le LLM. Il reçoit en entrée l'exigence structurée, les résultats d'analyse, et le cas échéant les observations vidéo. Plusieurs instances s'exécutent en parallèle grâce au mécanisme "Send()" de LangGraph, contrôlées par un "semaphore" "PLAN_CONCURRENCY". C'est à ce niveau que la mémoire à long terme est injectée : les préférences de l'utilisateur et les règles apprises enrichissent le prompt.
- **dispatch tc workers** : Ce nœud de synchronisation collecte les "blueprints" produits par les instances parallèles de "plan_single_req", puis les redistribue vers les "workers" de génération.
- **generate tc** : Chaque blueprint est transformé en cas de test complet par le LLM : préconditions détaillées, actions pas à pas, et résultats attendus avec des valeurs précises. Comme pour la planification, plusieurs workers opèrent en parallèle "GENERATE_CONCURRENCY".
- **synthesizer** : Reçoit l'ensemble des cas de test générés et effectue un traitement en trois passes : "deduplication" exacte (texte identique), deduplication floue (similarité sémantique au-delà d'un seuil de 75 %), et deduplication par recouvrement des résultats attendus. Ce filtrage assure que le livrable final ne contient pas de tests redondants.

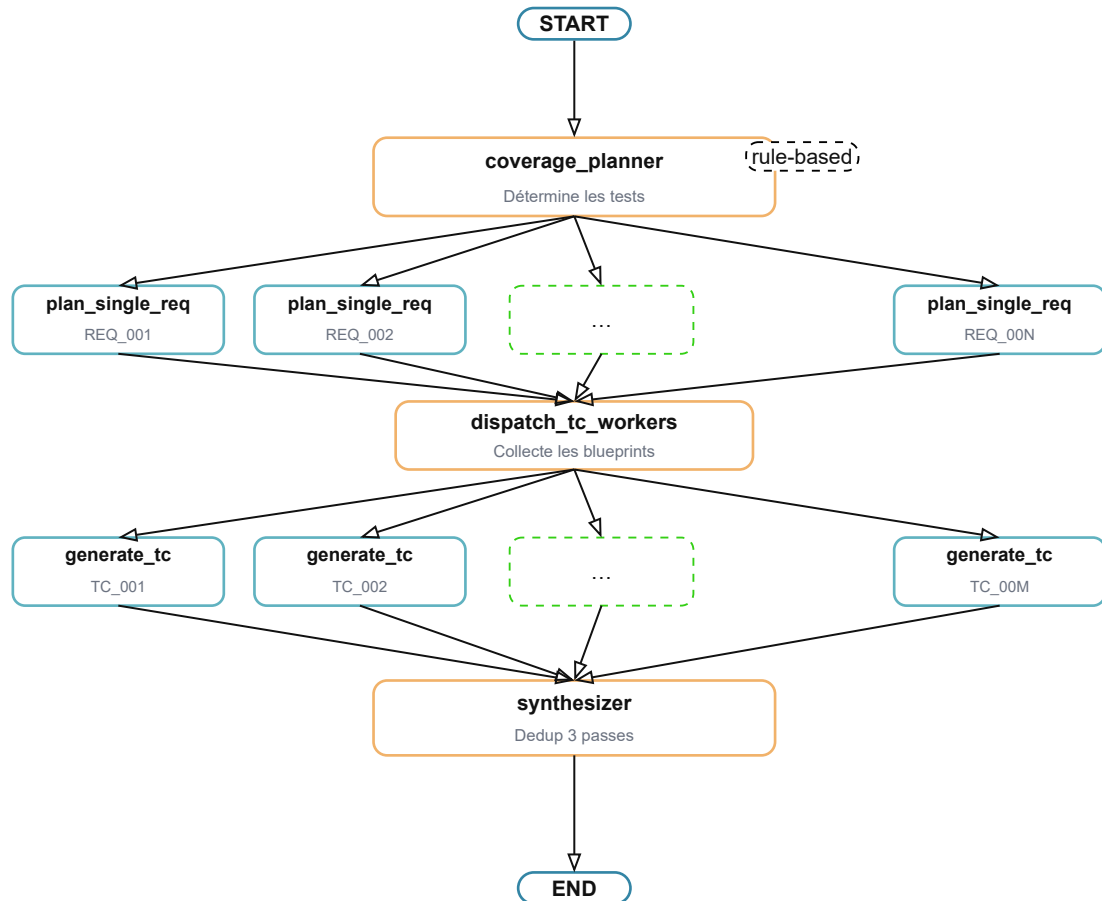


Figure 21: Agent 3 Workflows

3.3.6 Agent 4 : Évaluation et sortie

- **Evaluator** : Ce nœud constitue le gardien de qualité du système. Il opère en deux phases complémentaires.
 - La phase A applique des règles déterministes : détection de contradictions entre résultats attendus, vérification que chaque test reste dans le périmètre de l'exigence source, et contrôle de la précision des valeurs limites.
 - La phase B soumet les cas ayant passé la phase A à une évaluation par LLM, qui vérifie la cohérence globale, la pertinence, et la complétude. L'ensemble du processus garantit que cent pour cent des cas sont évalués.
- **Human review** : Ce nœud marque le point d'intervention humaine. Le pipeline se met en pause grâce à la fonction "interrupt()" de LangGraph et présente les résultats à l'utilisateur. L'exécution ne reprend que lorsque l'utilisateur a transmis ses décisions. Ce mécanisme est détaillé dans la section 3.5.

- **Process review** : Ce nœud interprète les décisions de l'utilisateur et oriente la suite du flux : vers la sortie si tout est approuvé, ou vers un nouveau cycle de génération si des cas ont été rejetés.
- **Output excel et Video output excel** : Ces nœuds produisent le livrable final au format Excel. Le nom du fichier inclut un numéro de version ("v1", "v2", "v3") qui s'incrémente à chaque cycle de revue, assurant la traçabilité des itérations.

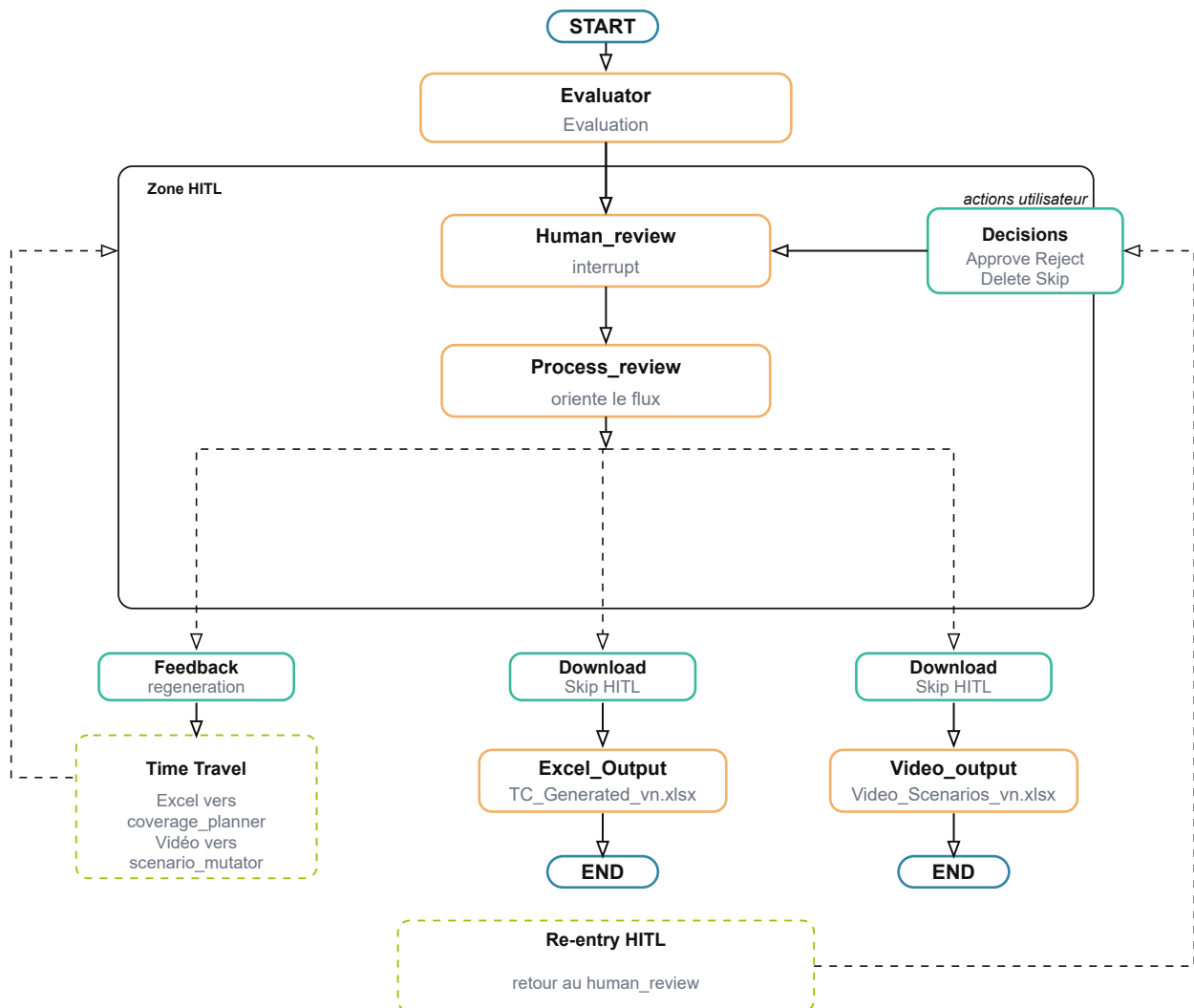


Figure 22: Agent 4 Workfolows

3.3.7 Parallélisme et contrôle de concurrence

Le pipeline exploite deux niveaux de parallélisme :

Le premier niveau utilise le mécanisme "**Send()**" de LangGraph pour distribuer le travail. Lorsque le planificateur identifie dix exigences à traiter, il crée dix instances parallèles de

"plan_single_req". Chaque instance opère de manière indépendante, avec son propre contexte et ses propres appels LLM.

Le second niveau intervient au sein des nœuds eux-mêmes. L'analyse des frames vidéo, par exemple, lance les appels LLM en parallèle via `"asyncio.gather()"`.

3.3.8 Déroulement temporel d'une exécution

Pour mieux apprécier l'enchaînement des échanges entre les différentes couches du système, le diagramme de séquence ci-dessous retrace le parcours complet d'une requête, depuis le chargement du fichier par l'utilisateur jusqu'à la mise en pause du pipeline pour revue humaine.

Séquence de génération des cas de test à partir d'un fichier Excel

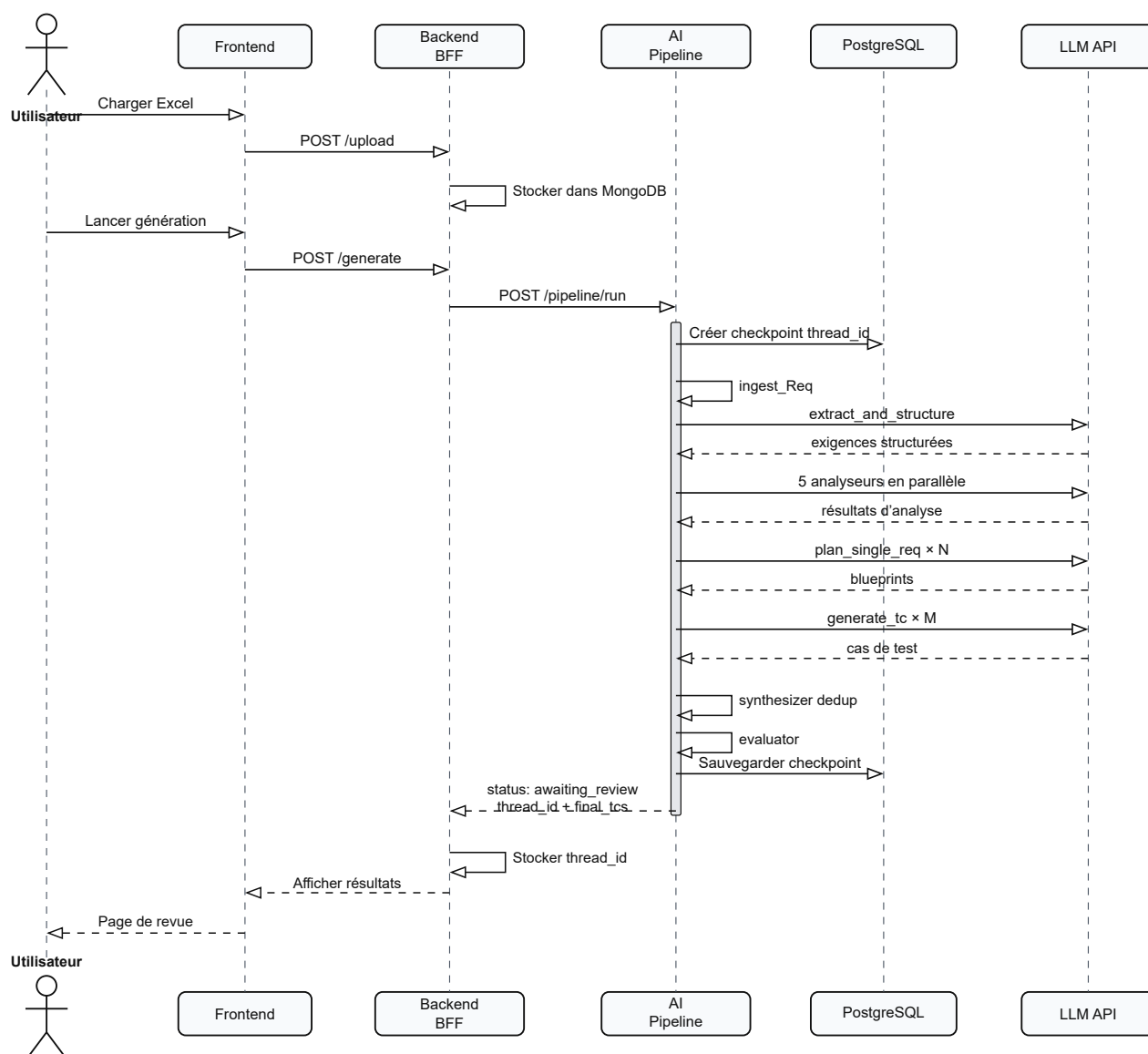


Figure 23: Diagramme de séquence du flux de génération Excel dans ADAS-R2T

3.4 Architecture "HITL" et "Time Travel"

L'une des contributions majeures de ce travail est l'intégration d'une boucle de contrôle humain directement dans le graphe d'exécution. Contrairement à une approche où l'utilisateur découvre les résultats une fois le traitement terminé, ici le pipeline s'interrompt volontairement pour solliciter l'avis de l'expert avant de poursuivre.

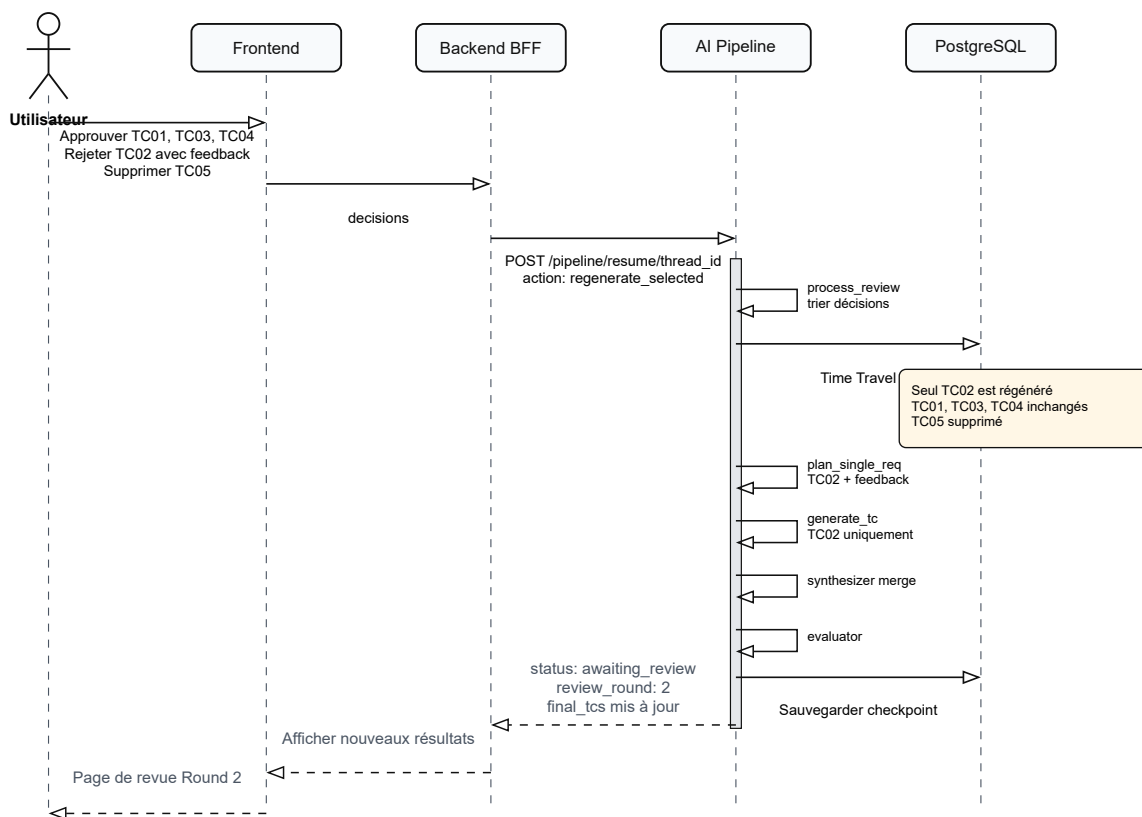


Figure 24: Revue HITL — Round 1 et régénération sélective

3.4.1 Le principe d'interruption

Le mécanisme repose sur la fonction "interrupt()" de LangGraph. Lorsque le pipeline atteint le nœud "human_review", il sauvegarde son état complet dans PostgreSQL et se met en pause. L'exécution ne reprend que lorsque l'utilisateur a transmis ses décisions via l'API. Ce comportement est rendu possible par le checkpointing, qui préserve l'intégralité du contexte entre la pause et la reprise.

Du point de vue de l'utilisateur, l'expérience est fluide : il reçoit les cas de test générés, les examine à son rythme, et soumet ses décisions. Du point de vue du pipeline, rien n'est perdu : lorsqu'il reprend, il dispose exactement du même état qu'au moment de la pause.

3.4.2 Les décisions de l'utilisateur

Pour chaque cas de test présenté, l'utilisateur dispose de trois actions possibles :

- **Approve** : le cas de test est validé et sera conservé tel quel dans le livrable final. Un commentaire optionnel peut être ajouté.
- **Reject** : le cas de test est insatisfaisant. L'utilisateur fournit obligatoirement un "feedback" expliquant ce qui doit être amélioré. Ce cas sera régénéré par le pipeline en tenant compte du "feedback".
- **Delete** : le cas de test est hors sujet ou redondant. Il sera supprimé définitivement du livrable.

Les cas de test pour lesquels l'utilisateur ne se prononce pas sont automatiquement considérés comme approuvés. Cette convention évite de contraindre l'utilisateur à examiner chaque élément lorsque la majorité des résultats est satisfaisante.

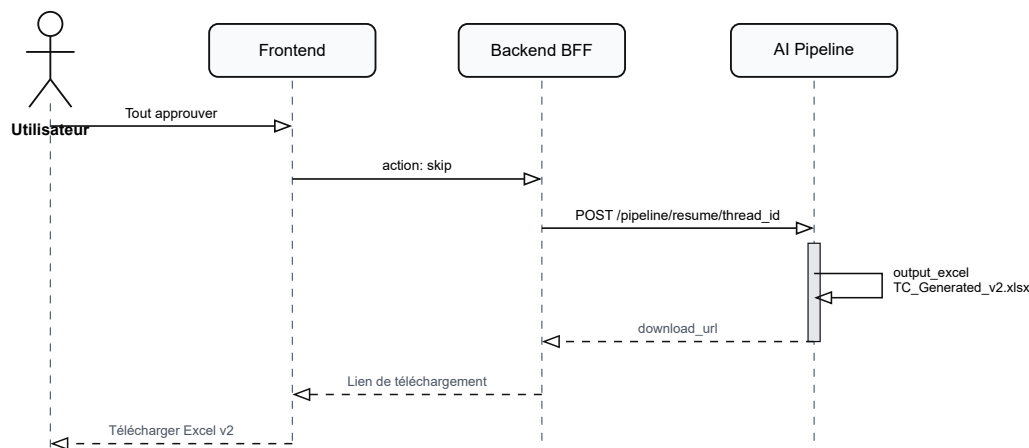


Figure 25: Revue HITL — approbation finale et génération du fichier Excel

3.4.3 Régénération sélective par "Time Travel"

Lorsque l'utilisateur rejette certains cas de test, le pipeline ne repart pas de zéro. Grâce au mécanisme de "Time Travel" de LangGraph, il revient au nœud "coverage_planner" en

conservant l'intégralité du contexte accumulé : exigences structurées, résultats d'analyse, "flow table", et observations vidéo.

Seuls les cas rejetés sont régénérés. Les cas approuvés restent rigoureusement inchangés : aucun appel LLM supplémentaire ne leur est consacré. Le planificateur reçoit les "feedbacks" de l'utilisateur et les intègre dans le prompt de génération, produisant ainsi des cas de test améliorés qui répondent spécifiquement aux remarques formulées.

Ce mécanisme présente un avantage considérable en termes de coût et de temps : régénérer deux cas de test sur vingt ne consomme qu'un dixième des ressources d'une régénération complète.

3.4.4 Régénération globale

L'utilisateur peut également demander une régénération de l'ensemble des cas de test, accompagnée d'un feedback global par exemple : « je veux des cas plus détaillés avec des valeurs limites plus précises ». Dans ce cas, le pipeline revient également au "coverage_planner", mais planifie la génération pour toutes les exigences en intégrant le feedback global dans chaque prompt.

3.4.5 Retour à HITL

Après le téléchargement du fichier Excel, l'utilisateur peut revenir à la page de revue pour lancer un nouveau cycle d'amélioration. Cette fonctionnalité utilise le mécanisme "aupdate_state()" de LangGraph pour repositionner le graphe au nœud "human_review", permettant une nouvelle itération sans relancer le pipeline depuis le début.

Chaque cycle de revue produit une nouvelle version du fichier ("v1", "v2", "v3"), assurant une traçabilité complète de l'évolution des résultats.

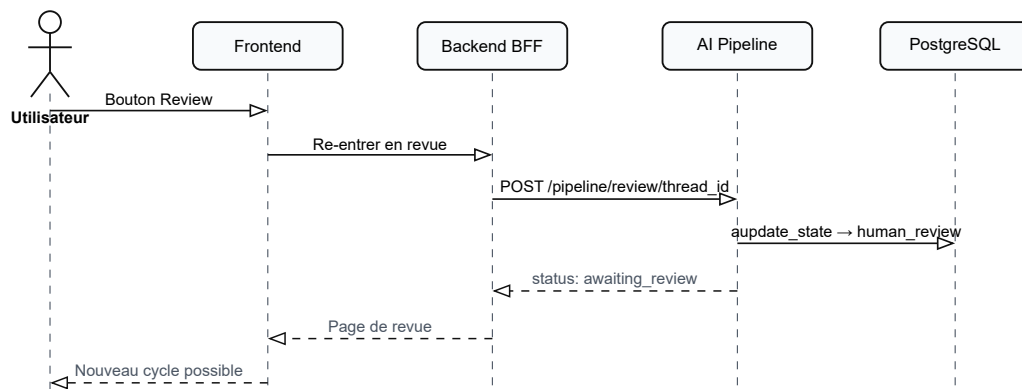


Figure 26: Revue HITL — réentrée optionnelle dans le cycle de revue

3.5 Architecture mémoire

Un système agentique qui ne retient rien de ses interactions passées reproduit les mêmes erreurs à chaque exécution. Pour dépasser cette limite, "ADAS-R2T" intègre une architecture mémoire à trois niveaux, chacun répondant à un besoin de persistance différent.

3.5.1 Les trois niveaux de mémoire

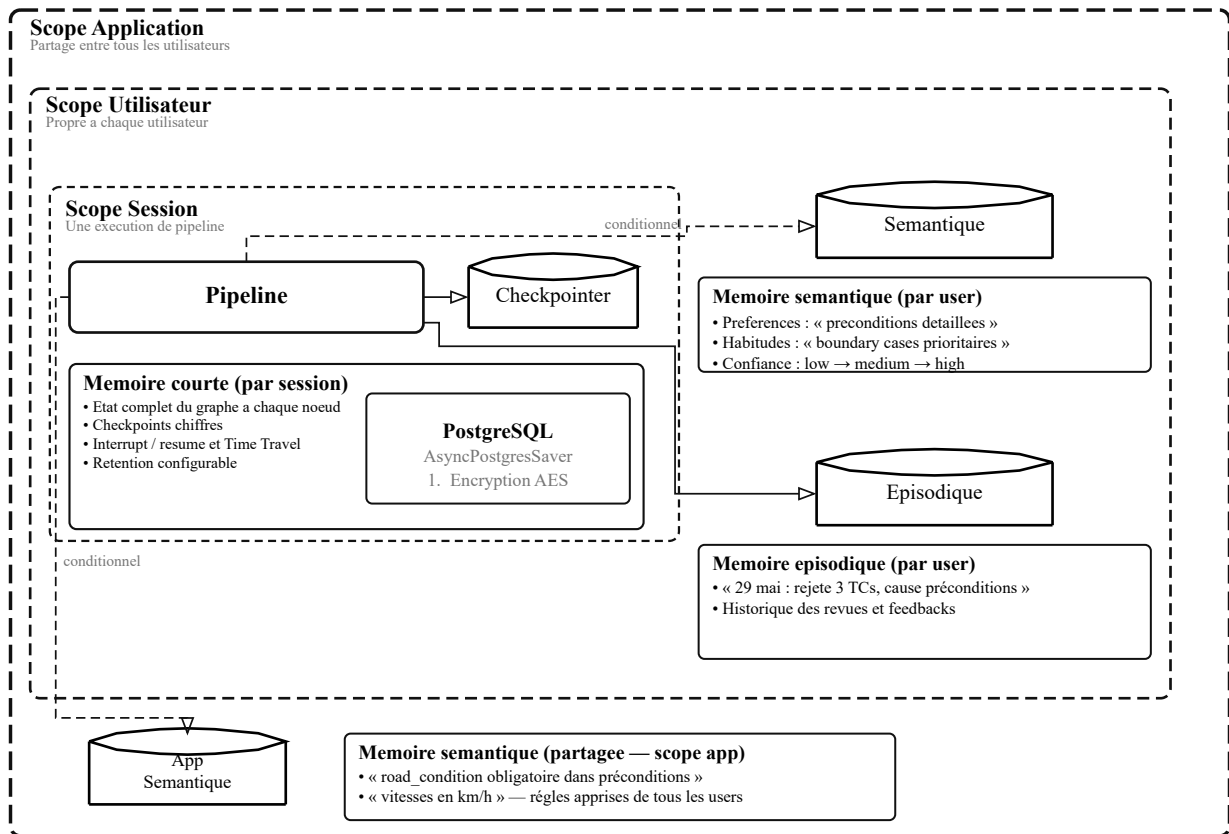


Figure 27: Architecture memoire

- **Mémoire de session (courte durée)** : La mémoire de session couvre une exécution unique du pipeline. À chaque nœud traversé, le checkpointer de LangGraph sauvegarde l'état complet du graphe dans PostgreSQL sous forme de checkpoint chiffré. Cet état inclut les exigences structurées, les résultats d'analyse, les cas de test générés, et les décisions de revue. Cette mémoire rend possibles trois mécanismes essentiels. L'interruption permet au pipeline de se mettre en pause au nœud de revue humaine et de reprendre exactement là où il s'était arrêté. Le "Time Travel" permet de revenir à un nœud antérieur pour régénérer des résultats sans perdre le contexte accumulé. La reprise après panne garantit que si le serveur redémarre en cours de traitement, le pipeline reprend au dernier checkpoint sauvegardé plutôt que de repartir de zéro. Les checkpoints sont chiffrés au repos par l'algorithme "AES", protégeant ainsi les données sensibles des exigences clients stockées dans la base. La durée de rétention est configurable via la variable "CHECKPOINT_CLEANUP_DAYS".
- **Mémoire utilisateur (longue durée)** : La mémoire utilisateur persiste au-delà des sessions individuelles. Elle capture les connaissances propres à chaque utilisateur et les réutilise lors des exécutions futures. Cette mémoire se décline en deux types :

- **La mémoire sémantique** : stocke les préférences et habitudes extraites des feedbacks de l'utilisateur. Par exemple, si un utilisateur rejette régulièrement des cas de test pour cause de préconditions insuffisantes, le système enregistre cette préférence et l'intègre automatiquement dans les prompts de génération lors des sessions suivantes. Chaque préférence est assortie d'un indice de confiance ("low", "medium", "high") qui croît avec la répétition.
- **La mémoire épisodique** : conserve l'historique factuel des revues effectuées par l'utilisateur : combien de cas ont été approuvés, rejetés, ou supprimés, et pour quelles raisons. Ce journal permet au système d'anticiper les attentes de l'utilisateur et d'adapter sa génération en conséquence.
- **Mémoire applicative (longue durée partagée)** : La mémoire applicative constitue le niveau le plus large. Elle stocke les règles de qualité qui transcendent les préférences individuelles et s'appliquent à l'ensemble des utilisateurs du système. Lorsqu'un feedback est reçu, un modèle de langage le classe automatiquement. Les remarques portant sur des standards du domaine (par exemple : « les vitesses doivent être exprimées en km/h ») sont orientées vers la mémoire applicative. Les préférences personnelles restent dans la mémoire utilisateur.

Une règle fondamentale gouverne la cohabitation entre ces deux niveaux : une connaissance présente dans la mémoire applicative n'est jamais dupliquée dans la mémoire utilisateur. Cette règle évite la redondance et garantit qu'une règle partagée est modifiée en un seul endroit.

- **Recherche sémantique et gestion du contexte** : La récupération des connaissances ne se fait pas de manière exhaustive. Lorsque le pipeline traite une exigence portant sur l'"ACC", il est inutile de lui rappeler des règles propres à l'AEB. La recherche s'appuie sur des embeddings vectoriels via pgvector pour identifier par similarité sémantique les connaissances pertinentes pour l'exigence en cours. Le volume de connaissances injectées dans le prompt est contrôlé dynamiquement. Le système calcule le budget disponible en fonction de la fenêtre de contexte du modèle utilisé et d'un ratio configurable "MEMORY_CONTEXT_RATIO". Si les connaissances récupérées dépassent ce budget, elles sont automatiquement résumées par un appel LLM avant injection, préservant l'essentiel sans saturer le contexte.

3.5.2 Flux d'écriture et de lecture

Le cycle de vie des connaissances suit deux chemins distincts :

Écriture : Après chaque revue humaine, le nœud "process_review" extrait les "feedbacks" des cas rejetés. Chaque feedback est classifié (applicatif ou utilisateur), vérifié contre les

doublons existants, puis stocké dans le niveau approprié. Un événement épisodique est simultanément enregistré.

Lecture : Avant chaque génération, le nœud "plan_single_req" interroge les trois niveaux de mémoire. Les connaissances pertinentes sont formatées en une section dédiée du prompt, précédée d'une instruction explicite : « applique toutes les règles et préférences ci-dessous ».

3.6 Architecture d'observabilité

Un système qui fait appel à des modèles de langage externes introduit une part d'imprévisibilité que les applications traditionnelles ne connaissent pas. Un prompt qui fonctionnait hier peut produire des résultats différents aujourd'hui. Un appel API peut échouer sans raison apparente. Pour maîtriser cette complexité, "ADAS-R2T" met en place trois piliers d'observabilité complémentaires.

3.6.1 Traçage LLM "Langfuse"

Langfuse est une plateforme open source spécialisée dans le suivi des applications basées sur des modèles de langage. Chaque appel LLM effectué par le pipeline est automatiquement tracé : le prompt envoyé, la réponse reçue, le nombre de tokens consommés, la durée de l'appel, et le modèle utilisé.

Ce niveau de détail permet d'identifier les prompts qui produisent des résultats insatisfaisants, de mesurer les coûts par exécution, et de comparer les performances entre différents modèles. Lorsqu'un cas de test généré est rejeté par l'utilisateur, l'équipe peut remonter dans Langfuse jusqu'au prompt exact qui l'a produit et comprendre pourquoi.

L'intégration est transparente : un "callback" "LangChain" enregistre automatiquement chaque interaction sans modifier le code des nœuds. La fonctionnalité s'active ou se désactive par simple configuration "LANGFUSE_ENABLED".

3.6.2 Métriques opérationnelles "Prometheus" et "Grafana"

Prometheus collecte les métriques quantitatives du système à intervalles réguliers. Un middleware instrumente chaque requête HTTP, et des compteurs dédiés suivent les indicateurs spécifiques au pipeline :

- Nombre de "pipelines" exécutés, par mode et par statut (terminé, échoué, en pause).
- Durée d'exécution de chaque nœud du graphe.
- Nombre et durée des appels LLM, par fournisseur et par modèle.
- Décisions HITL : nombre d'approbations, de rejets, et de suppressions.
- Opérations mémoire : écritures, lectures, et doublons évités.
- Taux d'erreur par nœud et par type d'exception.

"Grafana" consomme ces métriques et les restitue sous forme de tableaux de bord. Le dashboard « "ADAS-R2T Pipeline Monitor" » offre une vue en temps réel organisée en sections : vue d'ensemble, performance par nœud, utilisation LLM, activité HITL, opérations mémoire, et santé de l'infrastructure.

L'ensemble est déployé via "Docker Compose". "Node Exporter" collecte les métriques système ("CPU", mémoire, disque), tandis que "PostgreSQL Exporter" remonte les indicateurs de la base de données (connexions actives, temps de réponse des requêtes).

3.6.3 Logging structuré "structlog"

Le troisième pilier est le logging structuré. Contrairement aux logs textuels classiques, "structlog" produit des logs au format "JSON" où chaque information est un champ exploitable : nom du nœud, durée d'exécution, identifiant de session, nombre de résultats.

Un décorateur "**log_node**" enveloppe chaque nœud du graphe. Il enregistre automatiquement le début et la fin de l'exécution, la durée, et en cas d'erreur, le type d'exception et sa trace. Ce mécanisme ne nécessite aucune modification du code métier des nœuds : il suffit d'appliquer le décorateur.

Les logs structurés alimentent également les métriques Prometheus : le décorateur incrémente les compteurs de durée et d'erreurs à chaque exécution de nœud, assurant la cohérence entre les deux sources d'information.

3.7 Sécurité et résilience

Un système destiné à traiter des exigences fonctionnelles de sécurité automobile ne peut se permettre de négliger sa propre sécurité. Cette section présente les mécanismes mis en place pour protéger les données, garantir la disponibilité, et assurer l'isolation entre utilisateurs.

3.7.1 Flux d'écriture et de lecture

Le cycle de vie des connaissances suit deux chemins distincts :

Écriture : Après chaque revue humaine, le nœud "process_review" extrait les feedbacks des cas rejetés. Chaque feedback est classifié (applicatif ou utilisateur), vérifié contre les doublons existants, puis stocké dans le niveau approprié. Un événement épisodique est simultanément enregistré.

Lecture : Avant chaque génération, le nœud "plan_single_req" interroge les trois niveaux de mémoire. Les connaissances pertinentes sont formatées en une section dédiée du prompt, précédée d'une instruction explicite : « applique toutes les règles et préférences ci-dessous ».

3.8 Architecture d'observabilité

Un système qui fait appel à des modèles de langage externes introduit une part d'imprévisibilité que les applications traditionnelles ne connaissent pas. Un prompt qui fonctionnait hier peut produire des résultats différents aujourd'hui. Un appel API peut échouer sans raison apparente. Pour maîtriser cette complexité, "ADAS-R2T" met en place trois piliers d'observabilité complémentaires.

3.8.1 Traçage LLM "Langfuse"

Langfuse est une plateforme open source spécialisée dans le suivi des applications basées sur des modèles de langage. Chaque appel LLM effectué par le pipeline est automatiquement tracé : le prompt envoyé, la réponse reçue, le nombre de tokens consommés, la durée de l'appel, et le modèle utilisé.

Ce niveau de détail permet d'identifier les prompts qui produisent des résultats insatisfaisants, de mesurer les coûts par exécution, et de comparer les performances entre différents modèles.

Lorsqu'un cas de test généré est rejeté par l'utilisateur, l'équipe peut remonter dans Langfuse jusqu'au prompt exact qui l'a produit et comprendre pourquoi.

L'intégration est transparente : un callback LangChain enregistre automatiquement chaque interaction sans modifier le code des nœuds. La fonctionnalité s'active ou se désactive par simple configuration "LANGFUSE_ENABLED".

3.8.2 Métriques opérationnelles "Prometheus" et "Grafana"

Prometheus collecte les métriques quantitatives du système à intervalles réguliers. Un middleware instrumente chaque requête HTTP, et des compteurs dédiés suivent les indicateurs spécifiques au pipeline :

- Nombre de pipelines exécutés, par mode et par statut (terminé, échoué, en pause).
- Durée d'exécution de chaque nœud du graphe.
- Nombre et durée des appels LLM, par fournisseur et par modèle.
- Décisions HITL : nombre d'approbations, de rejets, et de suppressions.
- Opérations mémoire : écritures, lectures, et doublons évités.
- Taux d'erreur par nœud et par type d'exception.

Grafana consomme ces métriques et les restitue sous forme de tableaux de bord. Le dashboard « "ADAS-R2T Pipeline Monitor" » offre une vue en temps réel organisée en sections : vue d'ensemble, performance par nœud, utilisation LLM, activité "HITL", opérations mémoire, et santé de l'infrastructure.

L'ensemble est déployé via "Docker Compose". "Node Exporter" collecte les métriques système ("CPU", mémoire, disque), tandis que "PostgreSQL Exporter" remonte les indicateurs de la base de données (connexions actives, temps de réponse des requêtes).

3.8.3 Logging structuré "structlog"

Le troisième pilier est le logging structuré. Contrairement aux logs textuels classiques, "structlog" produit des logs au format "JSON" où chaque information est un champ exploitable : nom du nœud, durée d'exécution, identifiant de session, nombre de résultats.

Un décorateur "**log_node**" enveloppe chaque nœud du graphe. Il enregistre automatiquement le début et la fin de l'exécution, la durée, et en cas d'erreur, le type d'exception et sa trace. Ce

mécanisme ne nécessite aucune modification du code métier des nœuds : il suffit d'appliquer le décorateur.

Les logs structurés alimentent également les métriques Prometheus : le décorateur incrémente les compteurs de durée et d'erreurs à chaque exécution de nœud, assurant la cohérence entre les deux sources d'information.

3.9 Sécurité et résilience

Un système destiné à traiter des exigences fonctionnelles de sécurité automobile ne peut se permettre de négliger sa propre sécurité. Cette section présente les mécanismes mis en place pour protéger les données, garantir la disponibilité, et assurer l'isolation entre utilisateurs.

Point d'accès	Limite
Endpoints généraux	100 requêtes / minute
Pipeline (run, resume)	10 requêtes / heure
Authentification	20 requêtes / minute

Table 7: Limites de requêtes appliquées aux points d'accès de l'"API"

Ces limites sont appliquées par utilisateur et non globalement. Ainsi, un utilisateur qui atteint sa limite n'affecte pas les autres utilisateurs du système. La clé de limitation est dérivée de l'identifiant de l'utilisateur authentifié.

Séquence de génération des cas de test à partir d'un fichier Excel

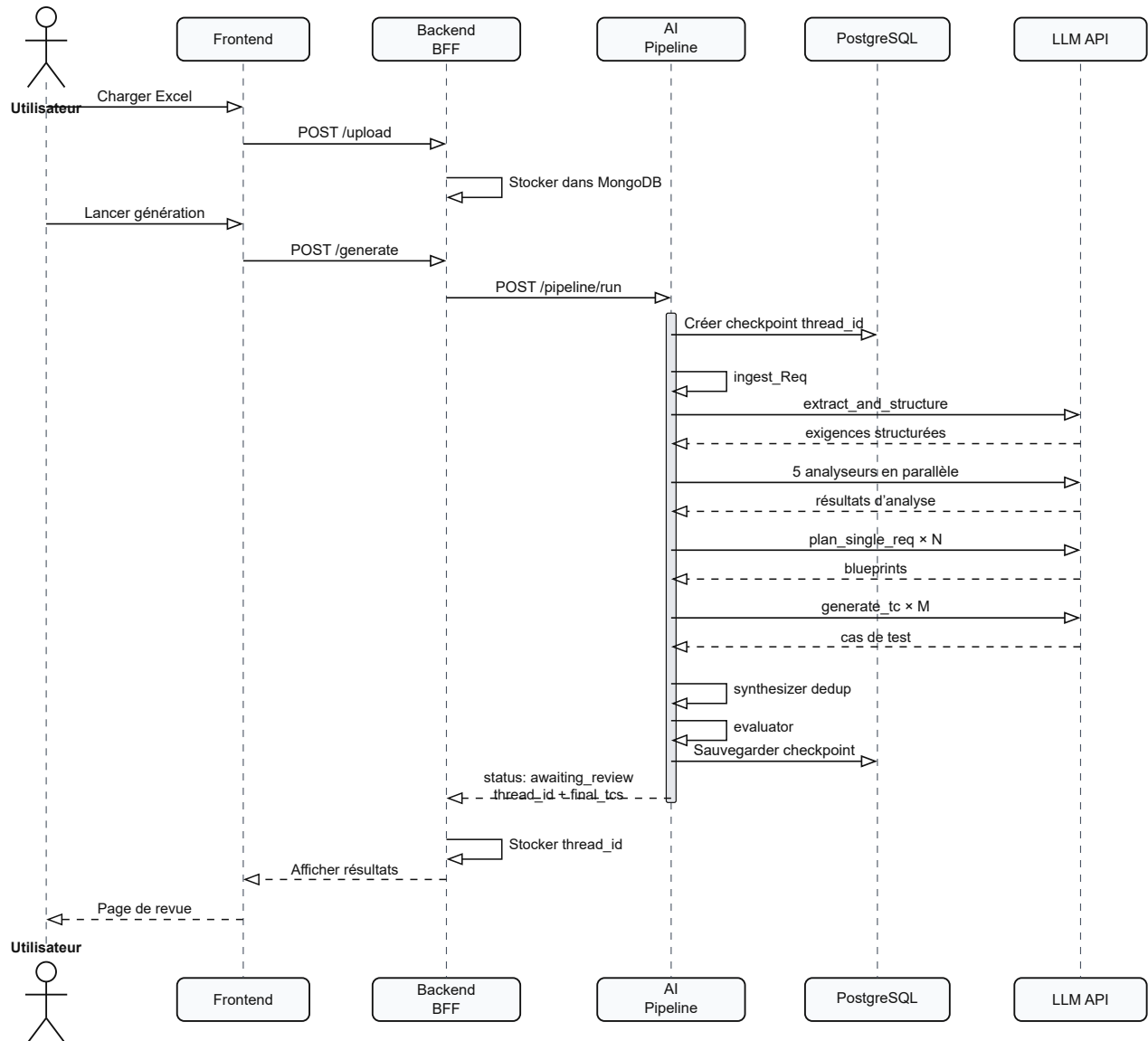


Figure 28: Diagramme de séquence du flux de génération Excel dans ADAS-R2T

Architecture Générale du Projet

4.1 Réalisation et implémentation

Ce chapitre quitte le terrain de la conception pour entrer dans celui de la construction. Il retrace les choix concrets de développement, l'organisation du code, les itérations successives du produit, et les obstacles rencontrés en cours de route.

4.1.1 Organisation du code source

Un projet de cette envergure **25 nœuds**, trois modes de fonctionnement, **plusieurs couches** d'infrastructure ne peut se permettre un code monolithique. Dès le départ, l'organisation du code a été pensée pour qu'un développeur puisse localiser n'importe quel composant en quelques secondes. Le projet adopte une structure "src/" qui isole le code applicatif des fichiers de configuration et d'infrastructure :

```

R2T-0.1V/
├─ docker/
│   ├── Dockerfile
│   ├── docker-compose.yml
│   ├── nginx/
│   ├── prometheus/
│   └─ grafana/
├─ src/
│   └─ app/
│       ├── main.py
│       ├── api/
│       │   └─ v1/
│       │       └─ pipeline.py
│       ├── core/
│       │   ├── config.py
│       │   ├── logging.py
│       │   ├── metrics.py
│       │   ├── checkpointer.py
│       │   ├── memory_store.py
│       │   ├── memory_manager.py
│       │   ├── middleware.py
│       │   ├── cleanup.py
│       │   └─ langgraph/
│       │       ├── graph.py
│       │       ├── utils.py
│       │       └─ nodes/
│       │           ├── ingest_excel.py
│       │           ├── extract_and_structure.py
│       │           ├── analyze_video.py
│       │           ├── video_frame_analyzer.py
│       │           ├── video_scenario_builder.py
│       │           ├── video_scenario_mutator.py
│       │           ├── coverage_planner.py
│       │           ├── plan_single_req.py
│       │           ├── generate_tc.py
│       │           ├── synthesizer.py
│       │           ├── evaluator.py
│       │           ├── human_review.py
│       │           ├── process_review.py
│       │           ├── output_excel.py
│       │           └─ video_output_excel.py
│       │       └─ prompts/
│       │           ├── extract_and_structure.md
│       │           ├── plan_single_req.md
│       │           ├── generate_tc.md
│       │           └─ ...
│       └─ schemas/
│           ├── workflow.py
│           └─ nodes/
│               ├── test_case.py
│               └─ video_scenarios.py
├─ tests/
├─ evals/
├─ .env
├─ pyproject.toml
└─ uv.lock

```

Listing 1: Arborescence du projet ADAS-R2T

4.1.2 Principes d'organisation

Trois principes gouvernent cette structure :

Un nœud, un fichier. Chaque nœud du graphe LangGraph est implémenté dans son propre fichier Python au sein du répertoire "nodes/". Le fichier "coverage_planner.py" contient exclusivement la logique du planificateur de couverture, "evaluator.py" celle de l'évaluateur, et ainsi de suite. Cette convention rend la navigation intuitive : pour comprendre le fonctionnement d'un nœud, il suffit d'ouvrir le fichier correspondant.

Séparation du code et des prompts. Les instructions envoyées aux modèles de langage ne sont pas noyées dans le code Python. Chaque prompt est stocké dans un fichier Markdown dédié, dans le répertoire "prompts/". La fonction utilitaire "load_prompt()" charge le contenu du fichier au moment de l'exécution. Cette séparation offre un double avantage : un ingénieur peut ajuster un prompt sans toucher au code, et chaque modification est traçable dans l'historique Git.

Schémas comme contrat. Le répertoire "schemas/" définit les structures de données qui circulent dans le pipeline. PipelineState dans "workflow.py" déclare l'ensemble des champs de l'état du graphe. Les schémas des nœuds ("test_case.py", "video_scenarios.py") définissent les formats de sortie attendus des appels LLM via Pydantic. Ces schémas servent à la fois de documentation et de validation automatique.

4.1.3 Rôles des répertoires

Répertoire	Rôle
<code>docker/</code>	Infrastructure de déploiement : Dockerfile, Compose, Nginx, Prometheus, Grafana.
<code>src/app/api/</code>	Points d'accès REST : endpoints du pipeline et authentification.
<code>src/app/core/</code>	Services transverses : configuration, logging, métriques, mémoire et checkpointing.
<code>src/app/core/langgraph/</code>	Cœur du pipeline : définition du graphe, orchestration des nœuds et gestion des prompts.
<code>src/app/core/langgraph/nodes/</code>	Les dix-neuf nœuds du graphe, organisés avec un fichier dédié par nœud.
<code>src/app/core/langgraph/prompts/</code>	Fichiers Markdown contenant les prompts utilisés par les appels LLM.
<code>src/app/schemas/</code>	Structures de données Pydantic : état du pipeline, cas de test et scénarios vidéo.
<code>tests/</code>	Tests unitaires et tests d'intégration réalisés avec pytest.
<code>evals/</code>	Évaluations de qualité, notamment avec DeepEval, à compléter selon les métriques retenues.

Table 8: Organisation logique des principaux répertoires du projet ADAS-R2T

4.1.4 Gestion des dépendances

Le projet utilise "uv" comme gestionnaire de paquets Python, choisi pour sa rapidité d'installation et sa compatibilité avec le format "pyproject.toml". Les dépendances sont

verrouillées dans "uv.lock", garantissant que chaque développeur et chaque conteneur Docker travaille avec les mêmes versions exactes.

Les dépendances se répartissent en plusieurs catégories : le cœur du pipeline (langgraph, langchain, fastapi), les connecteurs LLM (openai, langchain-google-genai), la persistance (psycpg, asyncpg), l'observabilité (langfuse, prometheus-fastapi-instrumentator, structlog), et le traitement multimédia (openpyxl, ffmpeg).

4.2 Itérations de développement

Le développement d'ADAS-R2T s'est déroulé en cinq itérations majeures, chacune ajoutant une couche de fonctionnalité au système. Cette progression reflète la méthodologie Design Science Research adoptée : concevoir un artefact, l'évaluer, puis l'enrichir en fonction des retours.

4.2.1 MVP 1 : Requirements to Tests

La première itération avait pour objectif de prouver le concept fondamental : un pipeline agentique peut-il générer des cas de test exploitables à partir d'exigences fonctionnelles ?

Ce premier pipeline comptait quatorze nœuds répartis en quatre étapes : ingestion du fichier Excel, analyse sémantique par cinq analyseurs parallèles, génération des cas de test, et export au format Excel. Le tout fonctionnait en mode synchrone — l'utilisateur lançait la génération et attendait le résultat sans indication de progression.

Les résultats étaient encourageants mais imparfaits. Le système produisait deux à trois cas de test par exigence, ce qui restait en deçà de la couverture attendue. Les prompts, rédigés en un seul bloc, souffraient d'*"attention dilution"* : le modèle, submergé par un contexte trop large, passait à côté de détails importants.

Mais l'essentiel était là : un fichier d'exigences entraînait, un fichier de cas de test sortait, en moins de deux minutes au lieu de plusieurs jours. Le concept était validé.

4.2.2 MVP 2 : Video Input Layer

La deuxième itération est née d'un constat : le texte seul ne suffit pas pour capturer la richesse des situations de conduite réelle. Un fichier d'exigences décrit ce que le système *doit* faire ; une vidéo montre ce qui *se passe vraiment* sur la route.

Quatre nouveaux nœuds ont été ajoutés pour former le pipeline vidéo. Le premier extrait les frames clés de la vidéo à intervalles réguliers et filtre les moments significatifs par détection de changement de scène. Le deuxième analyse chaque frame par un modèle multimodal, identifiant les véhicules, piétons, panneaux, et conditions environnementales. Le troisième reconstruit des scénarios complets en établissant des chaînes causales : cause, effet, conséquence. Le quatrième génère entre quinze et vingt-cinq mutations réalistes par scénario source, couvrant cinq stratégies de variation.

Cette capacité d'analyse vidéo avec raisonnement causal n'existait chez aucun outil concurrent identifié. Elle transforme quelques secondes de vidéo en dizaines de scénarios de test structurés. En parallèle, cette itération a intégré les retours d'une équipe de testeurs ADAS expérimentée. Dix catégories de problèmes récurrents ont été identifiées et transformées en règles de qualité : actions trop robotiques, préconditions incomplètes, valeurs limites vagues, tests hors périmètre, contradictions entre résultats attendus. L'évaluateur est devenu un gardien à deux phases (règles déterministes puis évaluation LLM), portant le taux d'évaluation de dix-sept à cent pour cent. Le streaming par "Server-Sent Events" et la planification par exigence (via "Send()") ont également été introduits dans cette itération, portant le nombre de cas générés de deux-trois à cinq-huit par exigence.

4.2.3 MVP 3 : Human in the Loop

La troisième itération a comblé le manque le plus critique des versions précédentes : le contrôle humain en cours d'exécution.

Jusqu'ici, le pipeline fonctionnait comme une boîte noire. Si trois cas de test sur vingt étaient insatisfaisants, il fallait tout relancer. Désormais, le pipeline se met délibérément en pause après l'évaluation. L'utilisateur examine chaque cas, approuve les bons, rejette les mauvais avec un feedback précis, et supprime les hors-sujet.

Le mécanisme de Time Travel de LangGraph permet ensuite de ne régénérer que les cas rejetés, sans reprendre le pipeline depuis le début. Le planificateur reçoit les feedbacks et les intègre dans les prompts, produisant des cas améliorés qui répondent spécifiquement aux remarques.

Cette boucle se répète jusqu'à satisfaction, chaque cycle produisant une nouvelle version du fichier ("v1", "v2", "v3").

L'infrastructure de production a également été posée dans cette itération : Docker Compose avec sept services (pipeline, PostgreSQL, Nginx, Prometheus, Grafana, exportateurs), checkpoints chiffrés en AES, rate limiting par utilisateur, isolation "multi-tenant", et tests automatisés avec pipeline CI via GitHub Actions.

4.2.4 MVP 4 : Chatbot

La quatrième itération, vise à ajouter un mode conversationnel au système. L'utilisateur pourra interroger le pipeline en langage naturel pour comprendre les résultats : « Pourquoi as-tu généré ce cas de test ? », « Explique-moi ce scénario vidéo », « Quels cas de test couvrent l'exigence "REQ_003" ? ».

Ce mode transformera ADAS-R2T d'un outil de génération en un véritable assistant intelligent, capable de justifier ses décisions et de guider l'ingénieur dans son travail de validation.

4.2.5 MVP 5 : Mémoire a long terme

La cinquième itération a doté le système d'une capacité d'apprentissage. Avant cette version, chaque exécution repartait de zéro, sans mémoire des interactions passées. Un utilisateur qui avait rejeté dix fois des cas de test pour cause de préconditions incomplètes devait reformuler la même remarque à chaque session.

La mémoire à long terme résout ce problème à trois niveaux. La mémoire sémantique utilisateur enregistre les préférences individuelles extraites des feedbacks. La mémoire sémantique applicative stocke les règles de qualité partagées par tous les utilisateurs. La mémoire épisodique conserve l'historique factuel des revues.

Un modèle de langage classe automatiquement chaque feedback : les remarques portant sur des standards du domaine sont orientées vers la mémoire applicative, les préférences personnelles restent au niveau utilisateur. Une règle stricte interdit la duplication entre les deux niveaux.

La recherche dans cette mémoire s'appuie sur des embeddings vectoriels (pgvector) pour ne récupérer que les connaissances pertinentes. Si le volume récupéré dépasse le budget de contexte du modèle, un résumé automatique condense l'essentiel avant injection dans le prompt.

4.2.6 Synthèse des itérations

Critère	MVP 1	MVP 2	MVP 3	MVP 4	MVP 5
Nœuds	14	19	25	25	25
Modes d'entrée	Excel	+Vidéo	Même	+Chat	Tous
Vidéo	Non	Oui	Oui	Oui	Oui
HITL	Non	Non	Oui	Oui	Oui
Mémoire	Non	Non	Non	Oui	Oui
Déploiement	Local	Local	Docker	VPS	VPS
Monitoring	Logs	+Langfuse	+Prometheus	+Grafana	Tous

Table 9: Comparaison synthétique des versions MVP du projet ADAS-R2T

4.3 Ingénierie des prompts

Dans un système agentique, la qualité des résultats dépend moins du code Python que des instructions données aux modèles de langage. Un prompt mal formulé produit des cas de test vagues ou hors sujet, quel que soit le raffinement de l'architecture qui l'entoure. Cette section décrit l'approche adoptée pour structurer, maintenir et enrichir les prompts du pipeline.

4.3.1 Pourquoi séparer les prompts du code

Dans la première version du système, les prompts étaient écrits directement dans le code Python, sous forme de longues chaînes de caractères imbriquées dans les fonctions des nœuds. Cette approche posait trois problèmes concrets.

Le premier était la lisibilité. Un prompt de deux cents lignes noyé dans du code Python devient rapidement illisible, surtout lorsqu'il contient des exemples, des contraintes, et des formats de sortie.

Le deuxième était la maintenabilité. Pour ajuster une formulation, il fallait ouvrir le fichier Python du nœud, trouver la chaîne de caractères, la modifier sans casser la syntaxe Python, puis relancer le serveur. Une opération qui devrait prendre trente secondes en prenait cinq minutes.

Le troisième était la traçabilité. Dans l'historique Git, les modifications de prompts se mêlaient aux modifications de code, rendant difficile la réponse à la question : « qu'est-ce qui a changé dans le prompt entre la version qui marchait et celle qui ne marche plus ? ».

La solution a été de déplacer chaque prompt dans un fichier Markdown dédié, stocké dans le répertoire "prompts/". Une fonction utilitaire "load_prompt(nom)" charge le contenu au moment de l'exécution :

```
1 def load_prompt(name: str) -> str:
2     """Charge un prompt depuis le repertoire prompts/."""
3     path = PROMPTS_DIR / f"{name}.md"
4     return path.read_text(encoding="utf-8")
```

Désormais, modifier un prompt revient à éditer un fichier "Markdown" pas besoin de toucher au code Python, pas de risque de casser la syntaxe, et chaque modification apparaît clairement dans l'historique Git.

4.3.2 Structure d'un prompt

Les prompts du système suivent une structure inspirée du framework "MISBAH", qui organise les instructions en sections clairement délimitées. Cette structure n'est pas arbitraire : elle reflète la manière dont les modèles de langage traitent l'information, en accordant plus d'attention au début et à la fin du contexte.

Un prompt typique se décompose en six sections :

Section 1 : Rôle et contexte. Le prompt commence par définir le rôle du modèle (« Tu es un ingénieur de test "ADAS" senior ») et le contexte métier (type de système, normes applicables). Cette mise en situation conditionne le registre et le niveau de détail des réponses. mmmmmmmmmmmmmmmmmmmmm

Section 2 : Tâche à accomplir. Une description précise de ce que le modèle doit produire, formulée de manière impérative : « Génère des blueprints de cas de test pour l'exigence suivante ».

mmmmmmmmmmmmmmmmmmmmmmmmmmmmmm

Section 3 : Données d'entrée. Les informations concrètes sur lesquelles le modèle doit travailler : le texte de l'exigence, les résultats d'analyse, la "flow table", et les observations vidéo le cas échéant.

mmmmmmmmmmmmmm

Section 4 : Format de sortie. Le schéma exact de la réponse attendue, généralement au format JSON ou sous forme de structure Pydantic. Cette section élimine l’ambiguïté et permet la validation automatique de la réponse.

mmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmm

Section 5 : Contraintes et règles. Les interdictions et obligations : « ne génère pas de cas hors du périmètre de l'exigence », « chaque précondition doit contenir une valeur numérique précise », « les actions doivent être formulées du point de vue du conducteur ». C'est ici que les retours des experts tests ont été intégrés sous forme de règles explicites.

mmmmmmmmmmmmmmmm

Section 6 : Exemples. Un ou deux exemples complets de la sortie attendue, montrant le niveau de détail et le style souhaités. Les exemples sont le moyen le plus efficace de communiquer les attentes au modèle.

4.3.3 L'enrichissement du contexte

Un prompt ne fonctionne pas seul. Sa puissance vient de ce qu'on lui injecte comme contexte. Au fil des itérations, le contexte fourni à chaque nœud s'est considérablement enrichi. Lors de la première version, le nœud "plan_single_req" recevait uniquement le texte brut de l'exigence. Le modèle devait deviner les transitions d'états, les contraintes temporelles, et les interactions utilisateur.

Dans la version actuelle, le même nœud reçoit un contexte composé de cinq couches :

Couche	Contenu
Exigence structurée	Texte, variables, conditions et seuils extraits des exigences fonctionnelles.
Résultats d'analyse	Transitions d'états, contraintes temporelles, interactions HMI et logique de calcul.
Flow table et acronymes	Table des transitions du système et acronymes extraits du fichier Excel source.
Observations vidéo	Scénarios causaux extraits de la vidéo de conduite dans le mode Excel + Vidéo.
Mémoire à long terme	Préférences de l'utilisateur, règles applicatives et historique des revues.

Table 10: Couches d'information exploitées par le pipeline ADAS-R2T

Chaque couche ajoute une dimension que le modèle n'aurait pas pu inférer seul. La "flow table", par exemple, révèle des transitions implicites que le texte de l'exigence ne mentionne pas. Les observations vidéo ancrent la génération dans des situations réelles. La mémoire à long terme évite de reproduire des erreurs déjà corrigées.

4.4 Déploiement

Faire tourner le système sur la machine d'un développeur est une chose. Le rendre installable, reproductible et opérable par quelqu'un d'autre en est une tout autre.

Le passage à **Docker** a constitué un tournant dans la maturité du projet : en une seule commande, l'ensemble de l'infrastructure sept services, trois bases de données, un **reverse proxy** et une pile de **monitoring** démarre et se configure automatiquement.

4.4.1 Le défi de la reproductibilité

Avant Docker, l'installation du système nécessitait une dizaine d'étapes manuelles : installer Python 3.12, créer un environnement virtuel, installer les dépendances, installer PostgreSQL, créer la base de données, activer l'extension pgvector, configurer les variables d'environnement, lancer le serveur. Si une seule étape était oubliée ou mal exécutée, le système refusait de démarrer, et le diagnostic pouvait prendre des heures.

Avec Docker, cette complexité disparaît derrière une seule **commande** :

```
1 cd docker
2 docker compose up -d
```

Shell

4.4.2 Le Dockerfile du pipeline

La construction de l'image Docker du pipeline suit une approche en couches, optimisée pour la vitesse de reconstruction :

```
1 FROM python:3.12-slim
2
3 # Dependances systeme (ffmpeg pour la video)
4 RUN apt-get update && apt-get install -y --no-install-recommends \
5     ffmpeg && rm -rf /var/lib/apt/lists/*
6
7 WORKDIR /app
8
9 # Dependances Python (changent rarement → cache Docker)
10 RUN pip install --no-cache-dir uv
11 COPY pyproject.toml uv.lock* ./
12 RUN uv sync --no-dev --no-editable
13
14 # Code applicatif (change souvent → dernière couche)
15 COPY src/ ./src/
16
17 ENV PYTHONPATH=/app/src
18 EXPOSE 8000
```

dockerfile

```

19
20 CMD ["uv", "run", "uvicorn", "app.main:app", \
21      "--host", "0.0.0.0", "--port", "8000"]

```

L'ordre des instructions n'est pas anodin. Les dépendances système et Python, qui changent rarement, sont installées en premier. Le code applicatif, qui change à chaque commit, est copié en dernier. Grâce au cache de Docker, une modification du code ne nécessite que la reconstruction de la dernière couche quelques secondes au lieu de plusieurs minutes.

4.4.3 L'orchestration des services

Le fichier "docker-compose.yml" déclare sept services qui forment l'écosystème complet du pipeline :

Service	Image	Port	Rôle
ai-pipeline	Build local	8000	Pipeline FastAPI + LangGraph
postgres	pgvector/pgvector:pg16	5432	Checkpoints et mémoire à long terme
nginx	nginx:alpine	80	Reverse proxy, support SSE et limite d'upload à 200 Mo
prometheus	prom/prometheus	9090	Collecte des métriques toutes les 15 secondes
grafana	grafana/grafana	3001	Dashboards de supervision en temps réel
node-exporter	prom/node-exporter	9100	Métriques système : CPU, mémoire et disque
postgres-exporter	prom/postgres-exporter	9187	Métriques PostgreSQL : connexions et requêtes

Table 11: Services Docker utilisés pour le déploiement et la supervision d'ADAS-R2T

Les dépendances entre les services sont gérées par Docker Compose. Le service ai-pipeline ne démarre qu'après la validation du **health check** de PostgreSQL, ce qui garantit la disponibilité de la base de données avant l'initialisation du pipeline. De son côté, Prometheus attend que le pipeline soit opérationnel avant de commencer la collecte des métriques. Cette orchestration permet d'éviter les erreurs de connexion au démarrage et d'assurer une mise en route plus fiable de l'environnement déployé.

4.4.4 La gestion des variables d'environnement

La configuration du système repose sur des variables d'environnement, conformément aux principes des applications *twelve-factor*. Aucun secret n'est codé en dur dans le code source. Deux fichiers sont utilisés pour gérer cette configuration. Le fichier .env.docker sert de modèle documenté avec des valeurs d'exemple, tandis que le fichier .env, ignoré par Git, contient les

valeurs réelles utilisées en développement ou en production. Cette approche améliore la sécurité, la portabilité et la reproductibilité du déploiement. Les variables se répartissent en plusieurs familles :

Famille	Exemples
LLM	content
Vidéo	content
Concurrence	content
PostgreSQL	content
Mémoire	content
Sécurité	content
Monitoring	content
Nettoyage	content

Table 12: Familles de variables d'environnement utilisées par ADAS-R2T

BIBLIOGRAPHIE

- [1] cumpusX, “Building effective agents.” [Online]. Available: <https://www.anthropic.com/engineering/building-effective-agents>
- [2] Anthropic, “Building effective agents.” [Online]. Available: <https://www.anthropic.com/engineering/building-effective-agents>
- [3] LangChain, “LangGraph Doc.” [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [4] Pythonation, “Misbah Framework for Contextual Engineering.” [Online]. Available: <https://github.com/Pythonation/misba7.git>
- [5] L. Birkemeyer and I. Schaefer, “Scenario Generation for Testing Automated Driving Systems.” 2025.
- [6] P. Ji *et al.*, “Txt2Sce: Scenario Generation for Autonomous Driving System Testing Based on Textual Reports.” [Online]. Available: <https://arxiv.org/abs/2509.02150>
- [7] X. Cai *et al.*, “Text2Scenario: Text-Driven Scenario Generation for Autonomous Driving Test.” [Online]. Available: <https://arxiv.org/abs/2503.02911>
- [8] A. Zorin and L. Mercier, “A New Approach to AD/ADAS Test Scenario Generation Using Open-Source Intelligence and Large Language Models.” 2024.
- [9] ASAM and IAV GmbH, “AI-powered ADAS Scenario Generation and Management.” [Online]. Available: <https://www.asam.net/application-stories/detail/ai-powered-adas-scenario-generation-and-management/>

ANNEXES

Texte provisoire.